

Large language model-empowered next-generation computer-aided engineering

Jiachen Guo^a, Chanwook Park^{b,c}, Dong Qian^{c,d}, Thomas J.R. Hughes^e, Wing Kam Liu^{b,c}

^aTheoretical and Applied Mechanics Program, Northwestern University, 2145 Sheridan Road, Evanston, 60201, IL, USA

^bDepartment of Mechanical Engineering, Northwestern University, 2145 Sheridan Road, Evanston, 60201, IL, USA

^cHIDENN-AI, LLC, 1801 Maple Ave, Evanston, 60201, IL, USA

^dDepartment of Mechanical Engineering, University of Texas, Dallas, 800 W. Campbell Road, Richardson, 75080, TX, USA

^eOden Institute for Computational Engineering and Sciences, University of Texas at Austin, 201 E 24th St, Austin, 78712, TX, USA

Abstract

Software development has entered a new era where large language models (LLMs) now serve as general-purpose reasoning engines, enabling natural language interaction and transformative applications across diverse domains. This paradigm is now extending into computer-aided engineering (CAE), offering a potential solution to the significant human effort and computational expense that constrain traditional finite element methods (FEM), particularly for large-scale parametric problems. Recent applications of LLMs in CAE have successfully automated routine tasks, including CAD model generation and FEM simulations. Nevertheless, these contributions, which primarily serve to reduce manual labor, are often insufficient for addressing the significant computational challenges posed by large-scale, high-dimensional systems. To this aim, we first introduce the concept of LLM-empowered CAE agent, where LLMs act as autonomous collaborators that plan, execute, and adapt CAE workflows. Then, we propose LLM-empowered CAE agent for data-free model order reduction (MOR), a powerful yet underused approach for ultra-fast large-scale parametric analysis due to the intrusive nature and labor-intensive redevelopment of solvers. LLMs can alleviate this barrier by automating derivations, code restructuring, and implementation, making intrusive MOR both practical and broadly accessible. To demonstrate feasibility, we present an LLM-empowered CAE agent for solving ultra-large-scale space-parameter-time (S-P-T) physical problems using Tensor-decomposition-based A Priori Surrogates (TAPS). Our results show that natural language prompts describing parametric partial differential equations (PDEs) can be translated into efficient solver implementations, substantially reducing human effort while producing high-fidelity reduced-order models. Moreover, LLMs can synthesize novel MOR solvers for unseen cases such as nonlinear and high-dimensional parametric problems based on their internal knowledge base. This highlights the potential of LLMs to establish the foundation for next-generation CAE systems.

Keywords:

Data-free model order reduction, Large language model, prompt engineering, Large-scale analysis, Parametric simulations

Highlight

- **CAE agents for autonomous CAE workflows:** LLM can act as proactive collaborators that plan, execute and adapt CAE workflows to save human labor and, more importantly, can learn from a growing knowledge database.
- **Overcoming barriers in intrusive model order reduction (MOR):** LLMs offer a path to automate the algebraic derivations and code restructuring required for data-free intrusive MOR, making this powerful but underused method more practical and accessible to CAE engineers.
- **Reducing LLM hallucination in data-Free MOR:** We show that a Chain-of-Thought prompting strategy, guided by curated examples, significantly reduces LLM hallucinations when deriving the complex mathematical equations required for intrusive, data-free MOR solvers.
- **Extrapolating solver capabilities beyond simple examples:** LLMs can develop sophisticated data-free MOR solvers (i.e., nonlinear/high-dimensional parametric problems) by correctly generalizing from provided simpler

linear examples, autonomously integrating advanced numerical schemes from their internal knowledge base.

- **Toward next-generation CAE systems:** LLM-empowered CAE agents can deliver surrogate models that achieve accuracy, speed, scalability and efficiency simultaneously, paving the way for future CAE platforms for engineering design and optimization.

1. Introduction

Software development is undergoing a paradigm shift, transitioning from explicit human-coded algorithms to systems defined by pre-trained neural networks [1]. In the latest phase of this evolution, the natural language itself serves as the programming interface, while large language models (LLMs) perform the underlying reasoning and implementation [2]. Within this new paradigm, developers leverage LLMs as general-purpose reasoning engines, defining objectives in natural language to generate the required implementation [3] automatically. The impact of this shift extends far beyond computer science: LLMs are already driving advances in biomedical research and healthcare [4], transforming educational tools and practices [5, 6], and augmenting expert work in high-stakes fields such as finance, consulting, and law [7]. Collectively, these trends signal a fundamental transformation in the way we harness computation across diverse domains.

The paradigm shift can also be projected to the domain of computer-aided engineering (CAE). Standard implementations of the finite element method (FEM), on commercial or open-source platforms, exemplify the traditional software paradigm. These frameworks rely on fixed, hard-coded numerical procedures to solve the governing equations for physical phenomena such as structural mechanics, heat transfer, and fluid dynamics [8]. The first wave of machine learning integration focused on improving these core numerical methods, using the universal approximation capability of neural networks to improve the accuracy and efficiency of FEM through adaptive meshing and basis functions [9, 10, 11, 12, 13, 14, 15, 16]. More recently, a second wave, driven by large language models (LLMs), is automating higher-level, labor-intensive processes, such as the creation of computer-aided design (CAD) models in standard CAE workflow [17, 18, 19]. In addition, recent attempts have focused on automating FEM simulations with legacy solvers. Hou et al. proposed AutoFEA, which reduces AI hallucinations by using a graph convolutional network transformer link prediction retrieval model [20]; Mudur et al. benchmarked the success rate of LLM agents in conjunction with COMSOL Multiphysics® [21]; Pandey et al. extended LLM capabilities through retrieval-augmented generation (RAG) for running simulations in OpenFOAM [22]. However, most of these applications only use LLMs as a function-calling tool to invoke standard CAE software, a task readily performed by human experts. In practice, this means that if a simulation fails to converge or produces physically unrealistic results, the system cannot independently diagnose the problem. For example, it does not know how to refine the mesh near a stress concentration or adjust the timestep to resolve a contact issue.

A CAE agent is an LLM-empowered system that acts as a proactive engineering collaborator, moving beyond the passive execution of predefined scripts. By leveraging an expandable “toolbox” of specialized functions described in natural language, CAE agents can autonomously plan and adapt entire analysis workflows, including designing experiments, running solvers, monitoring convergence, refining meshes, and post-processing. This allows it to intelligently automate the complex, trial-and-error decisions inherent in CAE, such as choosing a suitable material model or refining a mesh based on intermediate results. Therefore, CAE agents can capture and scale expert knowledge to greatly accelerate the design cycle by saving human labor.

Although CAE agents equipped with standard numerical methods, such as FEM, excel in optimizing the simulation workflow, a more fundamental challenge is the computational cost of the solvers themselves, especially for large-scale problems. This is where Model Order Reduction (MOR) tools offer another transformative solution. MOR techniques are typically classified into two categories: data-driven methods, including neural operators and other neural network-based surrogates [23, 24, 14, 15, 25]. Due to their non-intrusive nature, they can be applied broadly without modifying the original legacy solvers. However, they often require large amounts of offline simulation data as training data, which is prohibitive to generate for large-scale, high-dimensional parametric problems. In contrast, data-free MOR methods, including proper generalized decomposition [26] and the more recent Hierarchical Deep-learning Neural Network Tensor Decomposition (HiDeNN-TD) with its variants [10, 11, 13, 27], involve direct modifications to the solver’s core algorithms and are intrusive by nature. Although these methods do not require training data and offer

significant efficiency for extremely large-scale problems, their adoption is severely hindered by the effort required for extensive redevelopment of existing legacy codes, which poses a substantial barrier for most CAE engineers.

Recent breakthroughs in the mathematical reasoning and code generation abilities of LLMs offer a direct solution to this implementation barrier. State-of-the-art models have demonstrated expert-level performance in these domains [28, 29, 30], suggesting that they can automate the most labor-intensive aspects of data-free intrusive MOR, from initial algebraic derivations to complex restructuring and implementation of solver code. This represents a fundamental leap beyond previous standard CAE agents, which focused on automating the workflow using existing legacy solvers [17, 18, 19, 20, 21, 22]. In contrast, our proposed approach targets the core scientific challenge itself: the complex re-engineering of the solver architecture. Leveraging advanced, pre-trained language models thus provides a pathway to automate solver development, unlocking the full potential of data-free MOR with unprecedented efficiency while ensuring accuracy.

In this article, we introduce a novel CAE agent designed to automate the development of data-free intrusive MOR. Specifically, we focus on Tensor-decomposition-based A Priori Surrogates (TAPS) as the adopted data-free model [16] and show how natural language prompts can be translated into efficient solver implementations. The proposed approach significantly reduces human effort in the development of intrusive MOR models, including mathematical derivations, code implementation, and verification studies. As shown in Fig. 1, the resulting approach supersedes other tools across multiple performance metrics for large-scale, high-dimensional parametric problems. Through numerical examples, we illustrate the feasibility of this approach and discuss its potential as a foundation for future CAE systems that simultaneously meet the key performance requirements of accuracy, speed, resolution, memory efficiency, and scalability. This paper is structured as follows. First, we present the general concept of a CAE agent in Section 2. In Section 3, we introduce the basics of the adopted data-free intrusive MOR, i.e., Tensor-decomposition-based A Priori Surrogates (TAPS), and how LLMs can be leveraged to automate the development of TAPS for different parametric PDEs while overcoming LLMs’ hallucinations. Then we present the capability of the proposed framework via a variety of numerical examples in Section 4. Finally, in Sections 5 and 6, we discuss the potential improvements of the current approach in the near future.

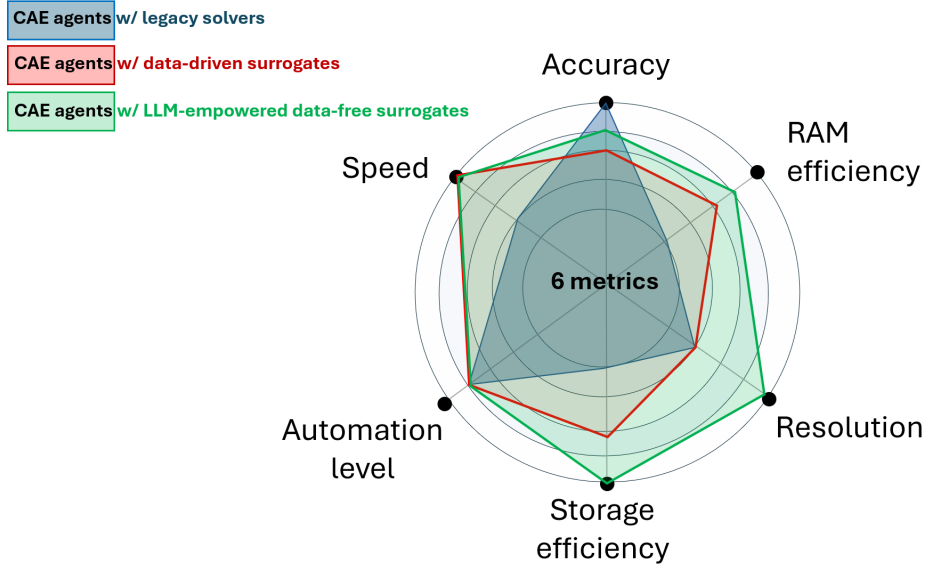


Figure 1: Comparison of different CAE agents with different tools in 6 performance metrics. CAE agents with legacy solvers have a high automation level and good accuracy, but suffer from computational cost for large-scale problems. CAE agents with data-driven surrogates can make predictions very fast once well trained, but the prediction accuracy is relatively limited. Training data-driven models can also be costly for large-scale, high-dimensional problems. CAE agents with LLM-empowered data-free MOR solvers, however, don’t require any offline training data. As a result, it has faster solving speed, better RAM and storage efficiency, and can easily handle large-scale problems that require a high-resolution mesh.

2. Preliminary of CAE agent

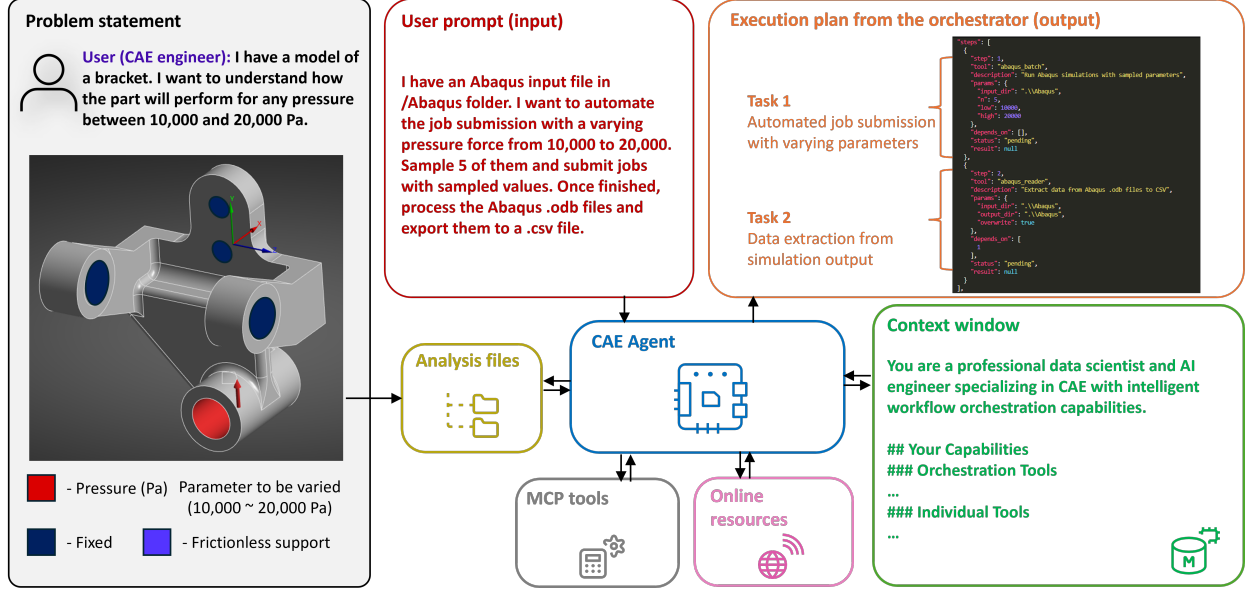


Figure 2: CAE agent for a data-driven surrogate modeling. The problem statement is provided in the left box. The user prompt describes the task more technically, and the execution plan from the orchestrator deciphers the user prompt, converting it into a series of executable model context protocol (MCP) tools.

In this section, we first introduce the general framework of a CAE agent: an autonomous system powered by a Large Language Model (LLM) and equipped with multiple CAE tools. The CAE agent is built by incorporating various modules with an LLM, where each module teaches the domain expertise of CAE to the LLM. As illustrated in Figure 2, the CAE agent consists of four key modules: analysis files, tools, online resources, and context window. To illustrate the roles of each module, we introduce a simple data-driven surrogate modeling task in CAE.

The problem statement is described in the left box of Figure 2. Consider a user who wants to create a surrogate model of a bracket subjected to a static pressure force acting on the bottom cylinder. This is a data-driven problem that involves a series of CAE tasks, including multiple job submissions with varying parameters, data extraction from the simulation output, data preprocessing, surrogate model training, and visualization. For an LLM to handle this CAE task, we first need to provide a working simulation file. For example, we can provide an input file of a commercial software (e.g., Abaqus) that reflects the problem definition, including the part geometry, boundary conditions, loading conditions, etc. Although we want to parameterize the loading condition (i.e., pressure force), we can leave it as a fixed number because the CAE agent can update it later on.

Another critical component for building a CAE agent is the context window, which defines how the LLM should behave as an engineering assistant. In this setup, the context window begins with the instruction: “You are a professional data scientist and AI engineer specializing in CAE and intelligent workflow orchestration capabilities”, as illustrated in Figure 2. This statement establishes the persona of the LLM as a CAE agent and specifies both the available tools and the general expectations of its role. The tools listed in the context window must correspond to those registered through the model context protocol (MCP) [31]. MCP serves as a universal adapter that enables LLMs to interact with various applications, tools, and data sources. In this problem setup, the MCP tools include automated job submission with varying parameters, data extraction from simulation output, surrogate model training, and visualization. Each MCP tool is provided with contextual guidance on when and how it should be used and what its capabilities are. A detailed explanation of how these MCP tools are built and connected to an LLM is provided in Appendix A. Finally, if relevant online resources are available, such as user documentation for CAE software, they can also be linked to the CAE agent to provide up-to-date references that support decision-making and workflow execution. This can be achieved through retrieval-augmented generation (RAG) [32].

The critical aspect of the CAE agent is that it acts as an autonomous agent capable of planning, executing, and adapting CAE workflows, rather than an automated function-calling tool for each subtask. Therefore, the CAE agent functions more like a proactive collaborator in engineering design and analysis, enabling faster exploration of parameter spaces and more efficient use of computational resources. To achieve this, we need to enhance both the context window and the MCP, allowing the CAE agent to plan and execute a series of tool calls. With this setup, once the user prompt is provided, the CAE agent converts it to an executable work plan that calls the available MCP tools. For example, as illustrated in Figure 2, the CAE agent converts the user prompt into an execution plan that calls two MCP tools: automated job submission and data extraction. Once the user approves, it will perform the two tasks automatically. The current demonstration is available for free to interested readers.¹

One possible critique of this workflow is that it resembles a conventional automation tool. This is a fair observation when dealing with relatively small problems that can be handled directly by human experts. However, the true transformative potential of the CAE agent lies in its scalability. As the domain expertise provided accumulates (i.e., analysis files, context window, MCP tools, and online resources) and the size of the problem becomes extremely large, the system eventually surpasses the cognitive capacity of a single human expert to fully manage. The development of a new intrusive data-free MOR tool exemplifies this threshold, a case that we will further elaborate on in the following sections of this article.

3. CAE agent for data-free model order reduction

3.1. Tensor-decomposition-based A Priori Surrogates (TAPS)

3.1.1. Tensor decomposition approximation

The computational modeling of many physical phenomena is challenged by their inherently multiscale and high-dimensional nature. Additive manufacturing, for instance, presents a formidable challenge to classical simulation tools such as the FEM because of the vast disparity in the scales involved. Resolving the micron-scale dynamics of the melt pool across the centimeter-scale geometry of the final component results in a huge number of degrees of freedom (DoFs) that are computationally intensive. Furthermore, the simulation constitutes a high-dimensional parametric problem, as the thermal field depends on numerous process/material parameters, such as laser spot size, power, scanning speed, laser absorptivity, and material diffusivity, which gives rise to the curse of dimensionality. To address such challenges, we adopt Tensor-decomposition-based A Priori Surrogates (TAPS) as the intrusive data-free MOR model [16]. TAPS is data-free since no training data is required to solve the unknowns in the surrogate model. Instead, we directly plug the approximation into the governing equation and solve the unknowns by minimizing the PDE residual.

We first introduce the basic formulation of TAPS. Assume a general D -dimensional space-parameter-time (S-P-T) problem with independent variables defined in Eq. 1:

$$\mathbf{x} = (\underbrace{x_1, \dots, x_S}_{\text{spatial variables}}, \underbrace{x_{S+1}, \dots, x_P}_{\text{parametric variables}}, x_t) \quad (1)$$

where $\mathbf{x}_s = (x_1, \dots, x_S)$ refer to spatial variables; $\mathbf{x}_p = (x_{S+1}, \dots, x_P)$ are parametric variables; x_t represents time. TAPS leverages tensor decomposition (TD) to approximate the solution field $u(\mathbf{x}_s, \mathbf{x}_p, x_t)$ as a sum of multiplication of univariate functions:

$$u^{TD}(\mathbf{x}_s, \mathbf{x}_p, x_t) = \sum_{m=1}^M \underbrace{[u_1^{(m)}(x_1) \cdots u_S^{(m)}(x_S)]}_{\text{spatial}} \underbrace{[u_{S+1}^{(m)}(x_{S+1}) \cdots u_P^{(m)}(x_P)]}_{\text{parametric}} \underbrace{u_t^{(m)}(x_t)}_{\text{temporal}} \quad (2)$$

where M represents the number of modes in TD.

Eq. 2 can be simplified using the following product notation:

$$u^{TD}(\mathbf{x}_s, \mathbf{x}_p, x_t) = \sum_{m=1}^M \prod_{d=1}^D u_d^{(m)}(x_d) \quad (3)$$

¹<https://hidennai.wordpress.com/>

where $u_d^{(m)}(x_d)$ is the univariate function for dimension d and mode m ; D is the total number of independent variables.

After decomposing the original high-dimensional multivariate function into a finite sum of multiplications of univariate functions to overcome the curse of dimensionality, each of these functions, $u_d^{(m)}(x_d)$, can then be parameterized using different approximation schemes, such as finite elements, radial basis functions, splines, or multilayer perceptrons (MLPs). In this paper, we use a novel AI-enhanced basis function called Convolution Hierarchical Deep-learning Neural Network (C-HiDeNN). This approach marries the merits of both locally supported finite element shape functions and the flexibility of machine learning. Consequently, C-HiDeNN not only maintains all the essential finite element approximation properties but also has controllable accuracy by virtue of the flexibility of neural networks. The detailed formulation of C-HiDeNN can be found in Appendix B. As a result, the C-HiDeNN-TD approximation of a general S-P-T problem can be written as:

$$u^{TD}(\mathbf{x}_s, \mathbf{x}_p, x_t) = \sum_{m=1}^M \prod_{d=1}^D \tilde{N}^{[d]}(x_d) \mathbf{u}_m^{[d]} \quad (4)$$

where $\tilde{N}^{[d]}(x_d)$ is the C-HiDeNN shape function for the d -th dimension; $\mathbf{u}_m^{[d]}$ refers to the discretized solution vector for m -th mode and d -th dimension.

3.1.2. Formulation of TAPS

As a data-free surrogate modeling approach, TAPS does not require offline training data to obtain the surrogate model. In contrast, TAPS solves the unknowns in the surrogate model $\mathbf{u}_m^{[d]}$ a priori using the generalized Galerkin projection. As a demonstration, we present the mathematical derivation for the following S-P-T PDE.

$$\frac{\partial u}{\partial t} - \frac{\partial}{\partial x} \left(\alpha \frac{\partial u}{\partial x} \right) = f(x, \alpha, t) \quad (5)$$

which is subject to the homogeneous Dirichlet boundary and initial conditions. This is a parametric PDE that models 1D transient heat transfer. The equation has 3 independent variables ($D = 3$), i.e., the spatial variable $x_s = x_1 = x$, the parametric variable $x_p = x_2 = \alpha$, and the temporal variable $x_t = x_3 = t$. For simplicity, we assume $f(x, \alpha, t) = f_x(x)f_\alpha(\alpha)f_t(t)$. The S-P-T weak form of this PDE can be written as:

$$\int_{\Omega} \delta u \nabla_t u dx d\alpha dt + \int_{\Omega} \nabla_x \delta u \cdot \alpha \nabla_x u dx d\alpha dt - \int_{\Omega} \delta u f(x, \alpha, t) dx d\alpha dt = 0 \quad (6)$$

where Ω is the S-P-T continuum. We define the following trial solution using C-HiDeNN-TD approximation.

$$u(x, k, t) = \sum_{m=1}^M [\tilde{N}^{[x]}(x) \mathbf{u}_m^{[x]}] [\tilde{N}^{[\alpha]}(\alpha) \mathbf{u}_m^{[\alpha]}] [\tilde{N}^{[t]}(t) \mathbf{u}_m^{[t]}] \quad (7)$$

The corresponding test function is obtained using the variational principle.

$$\begin{aligned} \delta u(x, \alpha, t) = & \underbrace{\sum_{m=1}^M [\tilde{N}^{[x]}(x) \delta \mathbf{u}_m^{[x]}] [\tilde{N}^{[\alpha]}(\alpha) \mathbf{u}_m^{[\alpha]}] [\tilde{N}^{[t]}(t) \mathbf{u}_m^{[t]}]}_{\text{spatial variation}} \\ & + \underbrace{\sum_{m=1}^M [\tilde{N}^{[x]}(x) \mathbf{u}_m^{[x]}] [\tilde{N}^{[\alpha]}(\alpha) \delta \mathbf{u}_m^{[\alpha]}] [\tilde{N}^{[t]}(t) \mathbf{u}_m^{[t]}]}_{\text{parametric variation}} \\ & + \underbrace{\sum_{m=1}^M [\tilde{N}^{[x]}(x) \mathbf{u}_m^{[x]}] [\tilde{N}^{[\alpha]}(\alpha) \mathbf{u}_m^{[\alpha]}] [\tilde{N}^{[t]}(t) \delta \mathbf{u}_m^{[t]}]}_{\text{temporal variation}}. \end{aligned} \quad (8)$$

Plugging Eqs. 7-8 into 6, we can obtain a nonlinear system of equations with respect to the generalized S-P-T DoFs $\mathbb{U} = (\mathbb{U}^{[x]}, \mathbb{U}^{[\alpha]}, \mathbb{U}^{[t]})$, where $\mathbb{U}^{[d]}$ denotes the vectorized solution vector for all modes in the d -th dimension $\mathbb{U}^{[d]} = [\mathbf{u}_1^{[d]}, \dots, \mathbf{u}_m^{[d]}]$. The subspace iteration algorithm is then leveraged to approximate the solution to the nonlinear equation by iteratively solving a series of linear problems in the subspace. Each linear subspace problem solves the following system of equations. Appendix C shows details of subspace iteration and how the corresponding matrix form is derived.

$$(\mathbf{A}^{[d]} \otimes \mathbf{K}_1^{[d]} + \mathbf{B}^{[d]} \otimes \mathbf{K}_2^{[d]}) \text{vec}(\mathbb{U}^{[d]}) = \text{vec}(\mathbb{Q}^{[d]}) \quad (9)$$

where $\text{vec}(\cdot)$ refers to vectorization; $\mathbf{K}_1^{[d]}, \mathbf{K}_2^{[d]}$ are sparse matrices related to spatial/parametric/temporal solution from the previous iteration; $\mathbf{A}^{[d]}, \mathbf{B}^{[d]}$ are coefficient matrices that depend on modal coefficients from other dimensions; $\otimes$ denotes the Kronecker product. The sparse matrices for each dimension are defined as follows:

Spatial matrices in dimension x :

$$\mathbf{K}^{[x]} = \mathbf{K}^{[B_x B_x]} = \mathbf{K}_1^{[x]} = \int_{\Omega_x} \tilde{\mathbf{B}}^{[x]T}(x) \tilde{\mathbf{B}}^{[x]}(x) dx \quad (\text{stiffness matrix}) \quad (10a)$$

$$\mathbf{M}^{[x]} = \mathbf{K}^{[N_x N_x]} = \mathbf{K}_2^{[x]} = \int_{\Omega_x} \tilde{\mathbf{N}}^{[x]T}(x) \tilde{\mathbf{N}}^{[x]}(x) dx \quad (\text{mass matrix}) \quad (10b)$$

where the C-HiDeNN shape function derivative $\tilde{\mathbf{B}}^{[x_d]}(x_d) = \frac{d\tilde{\mathbf{N}}^{[x_d]}(x_d)}{dx_d}$.

Parametric matrices in dimension α :

$$\mathbf{K}^{[N_\alpha \alpha N_\alpha]} = \mathbf{K}_1^{[\alpha]} = \int_{\Omega_\alpha} \tilde{\mathbf{N}}^{[\alpha]T}(\alpha) \alpha \tilde{\mathbf{N}}^{[\alpha]}(\alpha) d\alpha \quad (11a)$$

$$\mathbf{K}^{[N_\alpha N_\alpha]} = \mathbf{K}_2^{[\alpha]} = \int_{\Omega_\alpha} \tilde{\mathbf{N}}^{[\alpha]T}(\alpha) \tilde{\mathbf{N}}^{[\alpha]}(\alpha) d\alpha \quad (11b)$$

Temporal matrices in dimension t :

$$\mathbf{K}^{[N_t N_t]} = \mathbf{K}_1^{[t]} = \int_{\Omega_t} \tilde{\mathbf{N}}^{[t]T}(t) \tilde{\mathbf{N}}^{[t]}(t) dt \quad (12a)$$

$$\mathbf{K}^{[N_t B_t]} = \mathbf{K}_2^{[t]} = \int_{\Omega_t} \tilde{\mathbf{N}}^{[t]T}(t) \tilde{\mathbf{B}}^{[t]}(t) dt \quad (12b)$$

The coefficient matrices for each dimension can be derived as:

For dimension x :

$$\mathbf{A}^{[x]} = (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t N_t]} \mathbb{U}^{[t]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha \alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \quad (13a)$$

$$\mathbf{B}^{[x]} = (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t B_t]} \mathbb{U}^{[t]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \quad (13b)$$

where $\odot$ refers to the element-wise product.

For dimension α :

$$\mathbf{A}^{[\alpha]} = (\mathbb{U}^{[x]T} \mathbf{K}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t N_t]} \mathbb{U}^{[t]}) \quad (14a)$$

$$\mathbf{B}^{[\alpha]} = (\mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t B_t]} \mathbb{U}^{[t]}) \quad (14b)$$

For dimension t :

$$\mathbf{A}^{[t]} = (\mathbb{U}^{[x]T} \mathbf{K}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha \alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \quad (15a)$$

$$\mathbf{B}^{[t]} = (\mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \quad (15b)$$

The index form of the force vector $\mathbf{Q}^{[d]}$ for each dimension can be derived as:

For x -direction:

$$\mathbb{Q}_{n_x m}^{[x]} = \mathcal{Q}_{n_x}^{[x]} \cdot \left(\sum_{n_\alpha} u_{n_\alpha m}^{[x]} I_{n_\alpha}^{[x]} \right) \cdot \left(\sum_{n_t} u_{n_t m}^{[t]} I_{n_t}^{[t]} \right)$$

For t -direction:

$$\mathbb{Q}_{n_t m}^{[t]} = \mathcal{Q}_{n_t}^{[t]} \cdot \left(\sum_{n_x} u_{n_x m}^{[x]} I_{n_x}^{[x]} \right) \cdot \left(\sum_{n_\alpha} u_{n_\alpha m}^{[\alpha]} I_{n_\alpha}^{[\alpha]} \right)$$

For α -direction:

$$\mathbb{Q}_{n_\alpha m}^{[\alpha]} = \mathcal{Q}_{n_\alpha}^{[\alpha]} \cdot \left(\sum_{n_x} u_{n_x m}^{[x]} I_{n_x}^{[x]} \right) \cdot \left(\sum_{n_t} u_{n_t m}^{[t]} I_{n_t}^{[t]} \right)$$

where the subscript n_d refers to the number of discretized grid points in dimension d . The integration vectors are defined as:

$$\mathcal{Q}_{n_d}^{[d]} = \int_{\Omega_d} \tilde{N}_{n_d}^{[d]}(x_d) dx_d$$

$$I_{n_d}^{[d]} = \int_{\Omega_d} \tilde{N}_{n_d}^{[x_d]}(x_d) f_{x_d}(x_d) dx_d$$

As can be seen above, the final matrix form of the linear system in each subspace iteration is complicated. Deriving these matrices from the original S-P-T PDE presents the following challenges:

(1) **Complex Tensor Algebra:** Deriving the matrix form from the S-P-T weak form requires complex tensor algebra, including operations such as contraction, vectorization, and the Kronecker product. Although symbolic programming is useful for analytical derivations, its application to complex tensor algebra, particularly for high-order tensors, remains a significant challenge.

(2) **Susceptibility to Human Error:** The algebraic derivations for each subspace can differ significantly due to subtle distinctions between PDE terms, creating a high potential for human error. For example, the spatial variable x appears in both the time derivative and the diffusion term, but the corresponding coefficient matrices are different due to the subtle difference between the differential operators. Such nuances require detailed derivations for each term, increasing the likelihood of algebraic errors. Moreover, since there is an additional α in the diffusion term $\int_{\Omega} \nabla_x \delta u \cdot \alpha \nabla_x u dx d\alpha dt$, the resulting parametric matrix is, therefore, $K_{n_\alpha n'_\alpha}^{[N_\alpha \alpha N_\alpha]} = \int_{\Omega_\alpha} \tilde{N}_{n_\alpha}^{[\alpha]}(\alpha) \alpha \tilde{N}_{n'_\alpha}^{[\alpha]}(\alpha) d\alpha$ instead of $K_{n_\alpha n'_\alpha}^{[N_\alpha N_\alpha]} = \int_{\Omega_\alpha} \tilde{N}_{n_\alpha}^{[\alpha]}(\alpha) \tilde{N}_{n'_\alpha}^{[\alpha]}(\alpha) d\alpha$. This new weighted mass matrix is not a pre-built component in the standard FEM.

(3) **Lack of Generalizability:** The example problem illustrates a straightforward case that can be solved using TAPS. However, in more general scenarios involving additional parametric variables or new equation terms, the structure of the resulting matrix system is fundamentally altered. This alteration necessitates a complete re-derivation of the solver. Consequently, as an intrusive method, TAPS lacks the flexibility to easily adapt to different problem configurations.

(4) **Labor-Intensive Process:** The step-by-step mathematical derivation required by TAPS is not only intricate but also highly repetitive, demanding manual execution. For parametric PDEs, where the input space can include numerous material properties, forcing terms, or boundary conditions, this process is particularly inefficient. Consequently, any modification to these parameters requires the entire tedious derivation to be repeated from scratch. This manual re-derivation, coupled with the subsequent coding and verification effort, renders TAPS a labor-intensive and intrusive approach to model general parametric PDEs in CAE workflows.

Despite these challenges, we remark that the TAPS framework is inherently systematic and modular, as the entire process is defined by formal languages of mathematics and computer code. This structured nature makes TAPS an ideal candidate for automation using LLMs, which excel at both symbolic reasoning and code generation. Leveraging these capabilities can overcome the aforementioned challenges associated with the manual implementation of TAPS for various problems.

3.2. LLM-empowered mathematical reasoning for TAPS

While LLMs have achieved remarkable success in general language tasks, their performance deteriorates in specialized scientific domains such as data-free MOR. This deficit arises because the statistical, pattern-matching nature of LLMs is fundamentally misaligned with the rigorous derivation and numerical implementation required by the latest MOR techniques such as TAPS. Successfully applying these methods demands a precise sequence of symbolic and algebraic operations, from deriving weak forms to executing tensor contractions, which the probabilistic nature of a general-purpose LLM cannot guarantee.

Consequently, when applied to MOR solver development, LLMs often produce generic responses referencing well-established methods like Proper Orthogonal Decomposition (POD) [33], or generate code and equations that are syntactically invalid or conceptually flawed. This phenomenon is known as hallucination [34]. This capability gap renders general-purpose LLMs unreliable for direct use in sophisticated scientific workflows in CAE.

Various methods exist to mitigate LLM hallucination, including supervised fine-tuning (SFT), retrieval-augmented generation (RAG), and prompt engineering, with analogies in the domain of CAE shown in Fig. 3. SFT adapts a general-purpose LLM to a specific domain, but this process is often prohibitively expensive due to the need for vast amounts of high-quality training data. RAG enhances the LLM by retrieving information from a domain-specific database, though its reliability can be compromised if the retrieval process fetches irrelevant context. Prompt engineering, which involves crafting specialized prompts, is highly dependent on prompt quality, but does not require extensive domain-specific data. Given its cost-effectiveness and minimal data requirements, this paper employs a specific prompt engineering technique known as few-shot prompting to guide the LLMs [35].

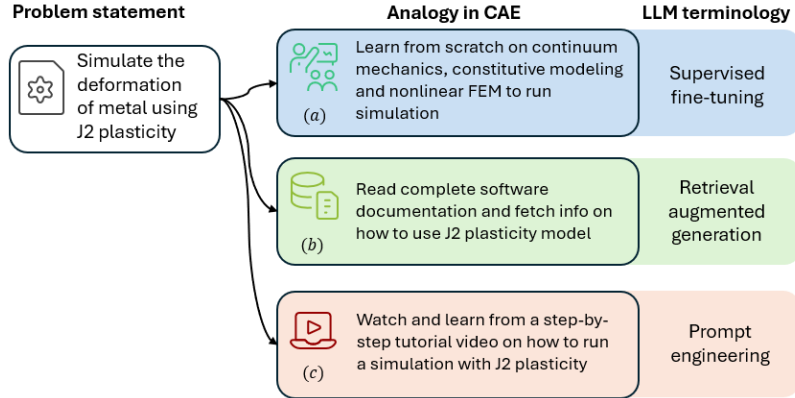


Figure 3: Countering LLM hallucinations with: (a) supervised fine-tuning (SFT): fine-tune the weights of LLMs by minimizing the prediction loss on domain-specific data. An analogy in CAE is to try to learn fundamental knowledge before using the simulation software. This process is the most expensive case. (b) retrieval augmented generation (RAG): fetch relevant information from the database to assist model prediction. An analogy is to directly look into specific documentations rather than learning from scratch (c) prompt engineering: embed a similar example directly in the prompt message sent to LLMs. An analogy is that, rather than searching the relevant document, directly learn a similar CAE example step-by-step.

3.2.1. Few-shot prompting

The few-shot prompt leverages the in-context learning capabilities of modern LLMs by providing a small number of demonstration examples, or “shots”, directly within the prompt. This approach requires no updates to the LLMs’ weights; instead, it guides LLMs to generate the desired output by establishing the necessary structural and syntactic patterns from the examples provided. Through this method, a general-purpose LLM can be adapted to produce highly structured domain-specific output, effectively bridging the gap between its generalized capabilities and the precise symbolic requirements within the TAPS framework. A template for this few-shot prompting approach is shown in Fig. 4, where the example template serves as a demonstration of the mathematical formulation. In this paper, we used the complete mathematical derivation of the TAPS solver for Eq. 5 as the few-shot example. The complete prompt is detailed in Appendix D, and the Langchain library is used for fine-grained control of LLMs [36].

Role-playing	You are a computational mechanics expert tasked with making targeted corrections to a mathematical derivation.
Example template	**TASK**: Create the mathematical derivation for the given problem statement based on the template example. **INPUT**: - Template example: <code>{{template_md_file}}</code> - Problem statement: <code>{{problem_statement_md_file}}</code> - Matrix definition: <code>{{matrix_md_file}}</code> **REQUIRED ACTIONS**: 1. Start with the complete template derivation 2. ... **IMPORTANT**: - Do the symbolic derivation step by step for every subspace iteration, especially the K direction iteration. - ...
Formatting guidelines	**OUTPUT FORMAT**: - Complete markdown file with the derivation - Use standard LaTeX math formatting ($\dots$ or $\dots$) - Preserve all original structure, headings, and content

Figure 4: Few-shot plotting example used for the mathematical reasoning prompt for TAPS development.

3.2.2. Chain-of-thought prompting

To handle the complex tensorial symbolic derivations required by TAPS, we enhance the example template using Chain-of-Thought (CoT) prompting [35]. Unlike the standard few-shot prompt, which only provides the final answer in its examples, CoT enriches the prompt with the intermediate, step-by-step reasoning needed to arrive at the solution. This distinction is illustrated in Fig. 5.

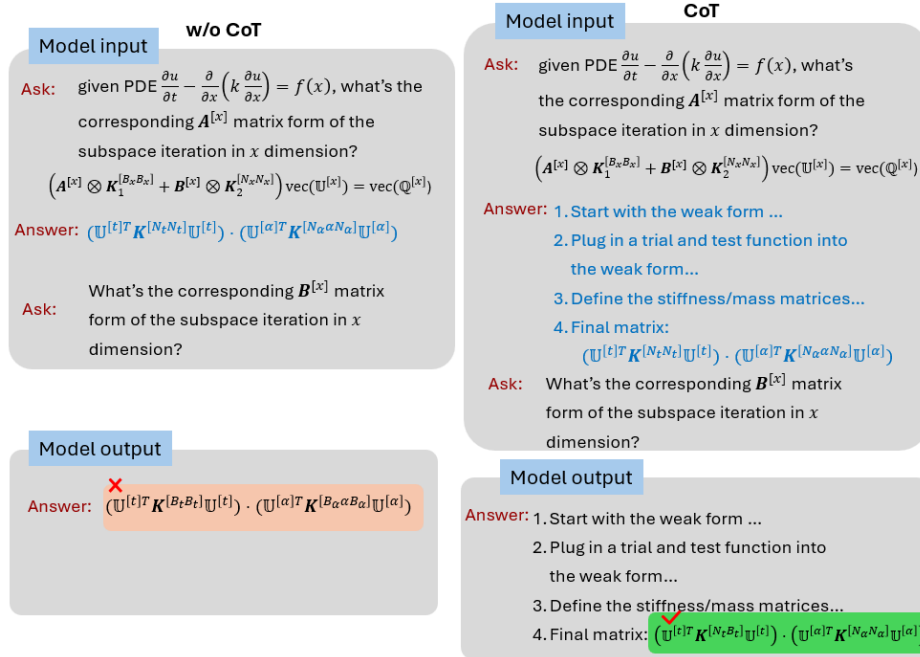


Figure 5: CoT can reduce the hallucinations of LLM in deriving the matrix form of each subspace iteration by teaching LLMs step-by-step.

As can be seen, LLMs tend to hallucinate when the few-shot examples directly output the final results rather than explaining the details of how the corresponding matrix form is derived. In contrast, the CoT prompt details each step on how the weak form is derived, followed by plugging the C-HiDeNN-TD approximation of the trial and test functions into the weak form. Then the required stiffness/mass matrices are defined before the final linear system of

equations is derived. By providing this explicit, step-by-step reasoning, the LLM learns to replicate the derivation process for new, unseen parametric PDEs.

From the above reasons, the provided CoT example plays a critical role in enabling the LLM to extrapolate the TAPS derivation to more general parametric PDEs. In this paper, we prepared the mathematical derivation of TAPS for Eq. 5 as the CoT example. The content, structured as shown in Fig. 6, is written in Markdown format.

1. Problem Setup and Governing Equation
2. Weak Form Derivation
3. Tensor Decomposition Framework
3. C-HiDeNN-TD Approximation
4. Subspace Iteration Concept
5. X-Direction Subspace Iteration - Complete Derivation
6. T-Direction Subspace Iteration - Complete Derivation
7. K-Direction Subspace Iteration - Complete Derivation
8. Matrix Assembly and Kronecker Products Physical
9. Interpretation and Computational Aspects

Figure 6: CoT example used in the current approach: detailed step-by-step mathematical reasoning process from original S-P-T PDE to the final matrix equations.

3.3. Automated agentic workflow

The development of CAE solvers consists of multiple procedures [37]. These include mathematical derivation, high-performance computational code implementation, and verification/validation/testing, where each step requires domain expertise and can be extremely arduous. With the help of LLMs, this labor-intensive development work can be significantly alleviated. For example, there are many powerful LLM-based coding assistants that have shown exceptional performance in a vast amount of coding benchmarks [38, 39]. These coding assistants can potentially accelerate the software development process.

Once the TAPS solver code for the new parametric PDE is implemented, the verification process can be automated through MCP tools, as shown in Fig. 7. This involves multiple stages. First, LLMs are called to generate a symbolic programming code to manufacture exact solutions to the corresponding PDE given suitable initial and boundary conditions. The LLMs then call this exact solution generator an MCP tool to generate the exact solution with its corresponding L^2 norm and forcing function. Subsequently, this information is inserted into a convergence study script to calculate the relative L^2 norm error. Finally, the LLM executes this script and automatically summarizes and plots the final results with the provided MCP tools.

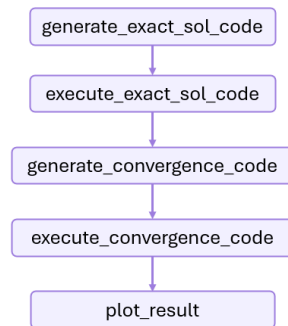


Figure 7: Automated agentic workflow for verification using different MCP tools.

As a result, LLMs can be used to automate the whole process from theory development, coding, and verification, as shown in Fig. 8. Although this agentic workflow can significantly accelerate the development, it should be noted that the inherently probabilistic nature of LLMs means that their outputs can be unreliable or deviate from the user's intent [40]. Consequently, human inspection is still essential for each procedure described in Fig. 8.

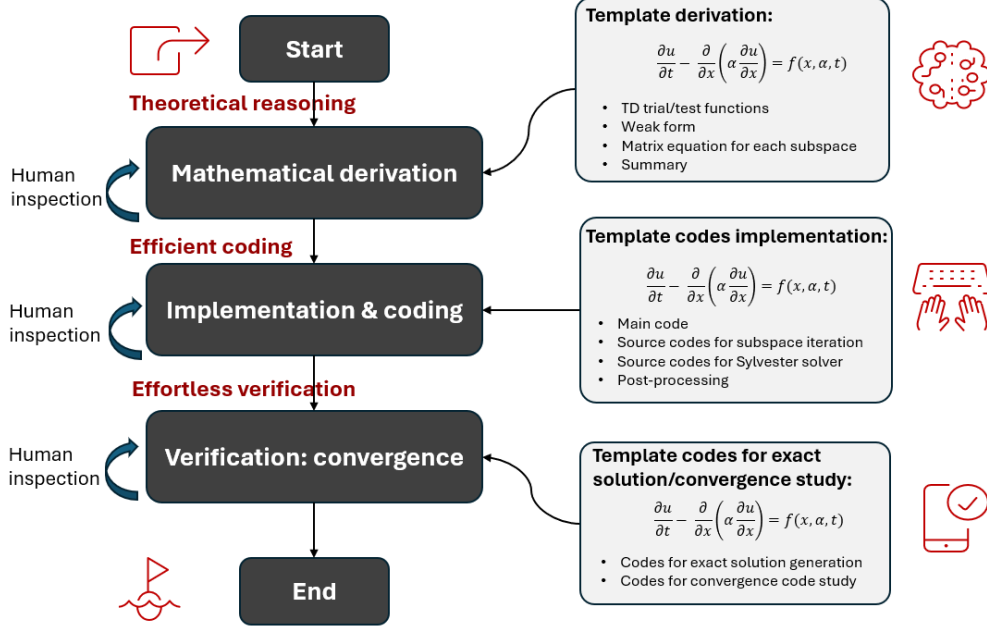


Figure 8: Complete agentic workflow for data-free TAPS development empowered by LLM.

4. Numerical results

In this section, we illustrate the LLM-empowered TAPS solver for different governing equations. For all the cases investigated in this section, GPT-5 is adopted as the LLM model. The 1D S-P-T problem shown in Eq. 5 is used as the original template problem, and the complete mathematical derivation, implementation, and convergence study codes are provided as templates in the crafted prompts. For all the numerical problems presented, mathematical derivation takes around 5 minutes; implementing numerical codes takes around 10 minutes; generating convergence study codes takes about 5 minutes. A single NVIDIA RTX A6000 GPU with 48 GB memory is used for the numerical study.

4.1. Magnetostatics

In this example, LLM is used to automatically derive the mathematical formulation of TAPS for the 3D magnetostatics equation. The numerical code is implemented using LLM according to the LLM-generated mathematical derivation. Finally, the convergence of the newly developed TAPS solver is investigated using the proposed automated workflow as shown in Fig. 7. The 3D magnetostatic equation in component form is as follows:

$$\begin{cases} \nabla^2 A_x(x, y, z) = -\mu_0 J_x(x, y, z) \\ \nabla^2 A_y(x, y, z) = -\mu_0 J_y(x, y, z) \\ \nabla^2 A_z(x, y, z) = -\mu_0 J_z(x, y, z) \end{cases} \quad (16)$$

where μ_0 represents the vacuum permeability; $\mathbf{J} = (J_x, J_y, J_z)$ refers to the current density vector. The solution field is approximated using C-HiDeNN-TD as follows:

$$A_x^{TD}(x, y, z) = \sum_{m=1}^M A_{xx}^{(m)}(x) A_{xy}^{(m)}(y) A_{xz}^{(m)}(z) \quad (17a)$$

$$A_y^{TD}(x, y, z) = \sum_{m=1}^M A_{yx}^{(m)}(x) A_{yy}^{(m)}(y) A_{yz}^{(m)}(z) \quad (17b)$$

$$A_z^{TD}(x, y, z) = \sum_{m=1}^M A_{zx}^{(m)}(x) A_{zy}^{(m)}(y) A_{zz}^{(m)}(z) \quad (17c)$$

Deriving the matrix form for Eq. 16 using the provided simple 1D S-P-T example presents two main challenges for an LLM. First, the target equation describes a vector field, whereas the provided example is a scalar PDE. Second, the Laplacian operator is applied to a 3D spatial field, not a 1D field as in the template.

Despite these complexities, with the given prompt, GPT-5 accurately derives the TAPS formulation in a single shot. This success is attributable to two key factors: GPT-5's ability to correctly derive the weak form for PDEs in a 3D spatial domain, and its understanding of essential tensor algebra concepts, including the Einstein summation convention, contraction, vectorization, and the Kronecker product. Consequently, with a well-formulated example template for a simpler PDE, GPT-5 can extrapolate the required mathematical concepts to a more complex problem.

Once the complete and detailed mathematical formulation of the TAPS solver is derived for the new PDE, the next step is to transform the final matrix equation in each subspace iteration into the corresponding code. To this end, GPT-5 is used again to generate the TAPS solver code. It should be noted that we do not ask GPT-5 to develop the new code from scratch. Instead, we provide the complete implementation of the 1D S-P-T TAPS code for Eq. 5 as the template. This can significantly facilitate code generation while minimizing potential errors. With the newly developed TAPS code, we investigate the convergence of the solver by leveraging a fully automated workflow in Fig. 7. To this end, the relative L^2 norm error is defined as shown below:

$$\epsilon_{L^2} = \frac{\|\mathbf{A}^{TD}(x, y, z) - \mathbf{A}^{\text{ex}}(x, y, z)\|_{L^2}}{\|\mathbf{A}^{\text{ex}}(x, y, z)\|_{L^2}} \quad (18)$$

The relative L^2 norm error and the computation time for different cases are plotted in Fig. 9. As can be observed from the figure, the convergence rate of the developed solver depends on the C-HiDeNN hyperparameter p . With a higher reproducing polynomial order p , the accuracy of the solution can be significantly improved. For the last case used in the convergence study, we use 1,000 grid points in each spatial direction, which is equivalent to 1 billion DoFs for a full-order model. Nevertheless, TAPS can solve this case in 8 seconds. This corroborates the significant speedup of TAPS as an MOR solver compared to standard full-order models such as FEM.

4.2. Linear elasticity

In this example, we use LLM to perform mathematical derivation, numerical implementation, and convergence study of the 3D linear elasticity problem. The governing equation is defined as:

$$\begin{cases} \mu \nabla^2 u(x, y, z) + (\lambda + \mu) \frac{\partial}{\partial x} [\nabla \cdot \mathbf{u}(x, y, z)] + f_x(x, y, z) = 0 \\ \mu \nabla^2 v(x, y, z) + (\lambda + \mu) \frac{\partial}{\partial y} [\nabla \cdot \mathbf{u}(x, y, z)] + f_y(x, y, z) = 0 \\ \mu \nabla^2 w(x, y, z) + (\lambda + \mu) \frac{\partial}{\partial z} [\nabla \cdot \mathbf{u}(x, y, z)] + f_z(x, y, z) = 0 \end{cases} \quad (19)$$

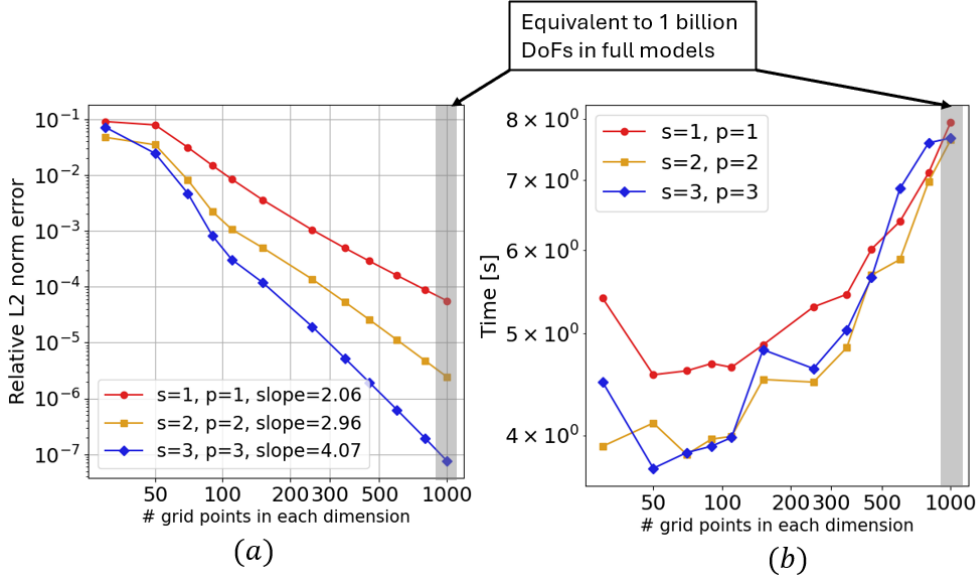


Figure 9: Convergence and computational time plots for the 3D magnetostatics.

where (λ, μ) represent Lamé parameters. The solution field is approximated with C-HiDeNN-TD:

$$u^{TD}(x, y, z) = \sum_{m=1}^M u_x^{(m)}(x) u_y^{(m)}(y) u_z^{(m)}(z) \quad (20a)$$

$$v^{TD}(x, y, z) = \sum_{m=1}^M v_x^{(m)}(x) v_y^{(m)}(y) v_z^{(m)}(z) \quad (20b)$$

$$w^{TD}(x, y, z) = \sum_{m=1}^M w_x^{(m)}(x) w_y^{(m)}(y) w_z^{(m)}(z) \quad (20c)$$

As can be readily seen, the main challenge in the mathematical derivation of LLMs for this case is the dilational term $(\lambda + \mu) \nabla [\nabla \cdot \mathbf{u}(x, y, z)]$, as it will result in coupled terms from different dimensions in the C-HiDeNN-TD approximation. However, the techniques for handling terms like this are not present in the template file of Eq. 5. Despite this, GPT-5 still manages to derive the final matrix form correctly. For example, the derived matrix form for the subspace iteration in u_y dimension is:

$$\left[\mu \cdot \mathbf{C}_{uu,lap}^{[y,x]} \otimes \mathbf{M}^{[y]} + \mu \cdot \mathbf{C}_{uu,lap}^{[y,y]} \otimes \mathbf{K}^{[y]} + (\mu \cdot \mathbf{C}_{uu,lap}^{[y,z]} + (\lambda + \mu) \cdot \mathbf{C}_{uu,div}^{[y]}) \otimes \mathbf{M}^{[y]} \right] \text{vec}(\mathbb{U}^{[y]}) = \text{vec}(\mathbb{Q}_{u_y}^{[y]}) \quad (21)$$

where the coefficient matrices are defined as:

$$\mathbf{C}_{uu,lap}^{[y,x]} = (\mathbb{U}^{[x]T} \mathbf{K}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[z]T} \mathbf{M}^{[z]} \mathbb{U}^{[z]}) \quad (22a)$$

$$\mathbf{C}_{uu,lap}^{[y,y]} = (\mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[z]T} \mathbf{M}^{[z]} \mathbb{U}^{[z]}) \quad (22b)$$

$$\mathbf{C}_{uu,lap}^{[y,z]} = (\mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[z]T} \mathbf{K}^{[z]} \mathbb{U}^{[z]}) \quad (22c)$$

$$\mathbf{C}_{uu,div}^{[y]} = (\mathbb{U}^{[x]T} \mathbf{K}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[z]T} \mathbf{M}^{[z]} \mathbb{U}^{[z]}) \quad (22d)$$

The forcing term is composed of 3 parts $\mathbb{Q}^{[y]} = \mathbb{Q}^{[y]} + \mathbb{Q}^{[y,uv]} + \mathbb{Q}^{[y,uw]}$ where $\mathbb{Q}^{[y]}$ is the integration of the multipli-

cation of the C-HiDeNN function and the forcing term, and the other two terms are defined as:

$$\mathbb{Q}^{[y,uv]} = -(\lambda + \mu) \mathbf{K}^{[y,mix]T} \mathbb{V}^{[y]} \mathbf{C}_{uv,div}^{[y]} \quad (23a)$$

$$\mathbb{Q}^{[y,uw]} = -(\lambda + \mu) \mathbf{M}^{[y]} \mathbb{W}^{[y]} \mathbf{C}_{uw,div}^{[y]} \quad (23b)$$

with two additional new coefficient matrices defined as:

$$\mathbf{C}_{uv,div}^{[y]} = (\mathbb{U}^{[x]T} \mathbf{K}^{[x,mix]} \mathbb{V}^{[x]}) \odot (\mathbb{U}^{[z]T} \mathbf{M}^{[z]} \mathbb{V}^{[z]}) \quad (24a)$$

$$\mathbf{C}_{uw,div}^{[y]} = (\mathbb{U}^{[x]T} \mathbf{K}^{[x,mix]} \mathbb{W}^{[x]}) \odot (\mathbb{U}^{[z]T} \mathbf{K}^{[z,mix]T} \mathbb{W}^{[z]}) \quad (24b)$$

As can be seen from the above equation, a new type of assembled matrix has been introduced.

$$\mathbf{K}^{[d,mix]} = \int_{\Omega_d} \tilde{\mathbf{B}}^{[d]T}(x_d) \tilde{\mathbf{N}}^{[d]}(x_d) dx_d \quad (25)$$

$\mathbf{K}^{[d,mix]}$ is resulted from dilational term in the weak form. Although it is not shown in the previous template derivation, GPT-5 manages to properly define this new matrix and derive the correct corresponding matrix form. This again manifests the mathematical reasoning capability and flexibility of using LLMs for MOR solver development. LLMs can extend the provided simple example to more complex cases based on their internal knowledge base.

With the newly developed TAPS code for elasticity, the convergence and speed of the solver are shown in Fig. 10. As expected, the convergence rate is controlled by the reproducing polynomial order p . When 1,000 grid points are used for each dimension (equivalent to 3 billion DoFs for a full-order model), the TAPS solver takes only about 10 seconds to complete the simulation.

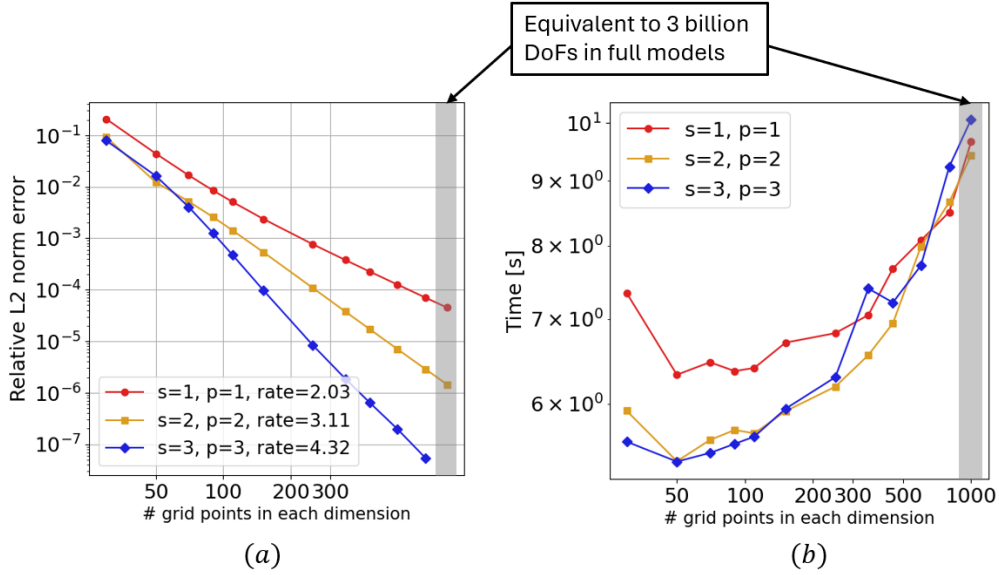


Figure 10: Convergence and computational time plots for the 3D elasticity.

4.3. Nonlinear PDE

In this example, we use LLMs to extend the original TAPS code for linear problems to a nonlinear case. The 3D transient diffusion-reaction equation is shown in Eq. 26.

$$\frac{\partial u(x, y, z)}{\partial t} - \nabla \cdot (\alpha \nabla u) + u^2 = f \quad (26)$$

Instead of solving a standard space-time problem, we treat diffusivity α as the parametric variable, and the S-P-T approximation to the parametric solution is:

$$u^{TD}(x, y, z, \alpha, t) = \sum_{m=1}^M u_x^{(m)}(x) u_y^{(m)}(y) u_z^{(m)}(z) u_\alpha^{(m)}(\alpha) u_t^{(m)}(t) \quad (27)$$

The main challenge for LLMs in this problem is the treatment of the nonlinear reaction term u^2 . Even though the TAPS solver has not been used to solve nonlinear PDEs before, when Eq. 11 is used as input, GPT-5 recognizes the nonlinearity of the problem and recommends using the fixed-point iteration method to address nonlinearity. This is because GPT-5 has a good understanding of general solution schemes to nonlinear PDEs and can actively adapt them to the current problem. As suggested by GPT-5, the linearized equation in each iteration can be written as:

$$\frac{\partial u^{(n+1)}}{\partial t} - \nabla_{x,y,z} \cdot (k \nabla_{x,y,z} u^{(n+1)}) + u^{(n)} u^{(n+1)} = f \quad (28)$$

where n is the index for the nonlinear iteration.

When GPT-5 was initially used for the mathematical derivation, it struggled to handle the integration of the linearized reaction term efficiently, as this form is atypical in the standard finite element:

$$\int_{\Omega} \delta u^{(i+1)} u^{(i)} u^{(i+1)} dx dy dz d\alpha dt \quad (29)$$

As a result, we manually defined a new type of matrix: a weighted mass matrix as input to the *matrix definition* part of the input prompt, as shown in Fig. 4.

$$\mathbf{M}^{[d](i)} = \int_{\Omega_d} \tilde{\mathbf{N}}^{[d]T}(x_d) u_d^{(i)}(x_d) \tilde{\mathbf{N}}^{[d]}(x_d) dx_d \quad (30)$$

where d refers to the dimension index.

With this new matrix definition, the GPT-5 model successfully derived the correct final matrix form for each subspace iteration of the TAPS solver. For example, the final matrix form of the subspace iteration in the z direction is derived as:

$$\left[\mathbf{C}_{x,z}^{[z]} \otimes \mathbf{K}^{[z]} + (\mathbf{C}_{x,x}^{[z]} + \mathbf{C}_{x,y}^{[z]} + \mathbf{C}_t^{[z]}) \otimes \mathbf{M}^{[z]} \right] \text{vec}(\mathbb{U}^{[z]}) + \left[\sum_{p=1}^M (\mathbf{\Gamma}^{[z](p)} \otimes \mathbf{M}^{[z](p)}) \right] \text{vec}(\mathbb{U}^{[z]}) = \text{vec}(\mathbb{Q}^{[z]}) \quad (31)$$

where the coefficient matrices are defined as:

$$\mathbf{C}_{x,z}^{[z]} = (\mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[y]T} \mathbf{M}^{[y]} \mathbb{U}^{[y]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha \alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \odot (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t N_t]} \mathbb{U}^{[t]}) \quad (32a)$$

$$\mathbf{C}_{x,x}^{[z]} = (\mathbb{U}^{[x]T} \mathbf{K}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[y]T} \mathbf{M}^{[y]} \mathbb{U}^{[y]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha \alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \odot (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t N_t]} \mathbb{U}^{[t]}) \quad (32b)$$

$$\mathbf{C}_{x,y}^{[z]} = (\mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[y]T} \mathbf{K}^{[y]} \mathbb{U}^{[y]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha \alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \odot (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t N_t]} \mathbb{U}^{[t]}) \quad (32c)$$

$$\mathbf{C}_t^{[z]} = (\mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]}) \odot (\mathbb{U}^{[y]T} \mathbf{M}^{[y]} \mathbb{U}^{[y]}) \odot (\mathbb{U}^{[\alpha]T} \mathbf{K}^{[N_\alpha \alpha N_\alpha]} \mathbb{U}^{[\alpha]}) \odot (\mathbb{U}^{[t]T} \mathbf{K}^{[N_t B_t]} \mathbb{U}^{[t]}) \quad (32d)$$

This example shows that human intervention is still required for some unseen, complicated mathematical operations. Similarly, the convergence and computation cost plots are shown in Fig. 11. The convergence rates for different C-HiDeNN hyperparameters follow the same pattern. When 1,000 grid points are used for each dimension, the total equivalent DoFs for a full-order model are 1 billion. However, the nonlinear TAPS solver only takes around 27 seconds, which is much faster than the corresponding full-order models such as FEM.

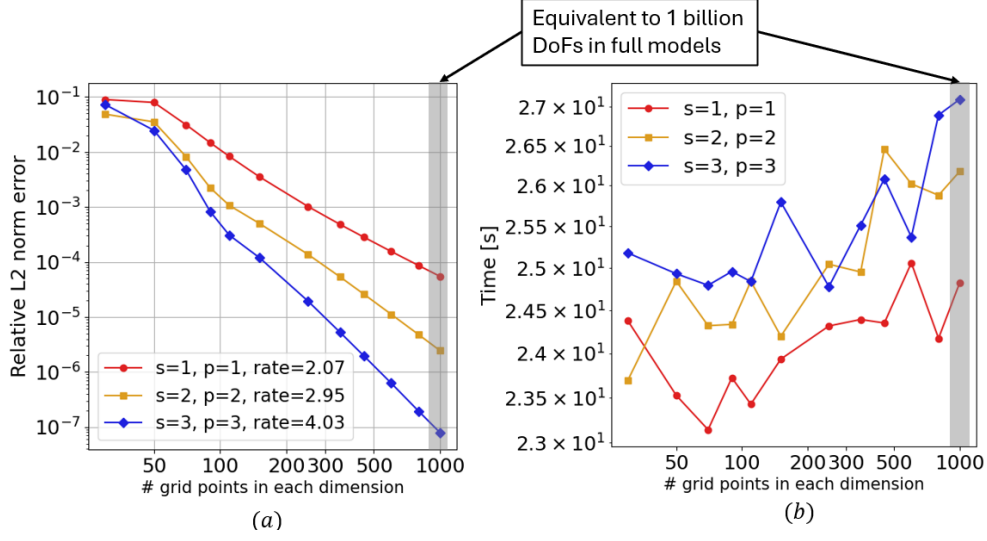


Figure 11: Convergence plot for the 3D nonlinear PDE problem.

4.4. Transient heat transfer with spatially variant diffusivity

In this example, our aim is to leverage LLMs to develop the TAPS solver for the following problem:

$$\frac{\partial u[\mathbf{x}, \alpha(\mathbf{x}), t]}{\partial t} - \nabla \cdot [\alpha(\mathbf{x}) \nabla u[\mathbf{x}, \alpha(\mathbf{x}), t]] = f \quad (33)$$

where the diffusivity field $\alpha(\mathbf{x})$ can vary at different locations. As shown in Fig. 12, the whole domain is decomposed into $D_x \times D_y \times D_z$ subdomains where the diffusivity is homogeneous in each subdomain but can arbitrarily change within a predefined range $[\alpha_{min}, \alpha_{max}]$. The parametric input variable $\alpha = (\alpha_1, \alpha_2, \dots, \alpha_i, \dots, \alpha_{D_x D_y D_z})$ where α_i stands for the diffusivity for the i -th subdomain. Therefore, the solution to this problem is a S-P-T field and can be approximated using TD as follows:

$$u^{TD}(x, y, z, \alpha_1, \alpha_2, \dots, \alpha_{D_x D_y D_z}, t) = \sum_{m=1}^M u_x^{(m)}(x) u_y^{(m)}(y) u_z^{(m)}(z) \left[\prod_{i=1}^{D_x D_y D_z} u_{\alpha_i}^{(m)}(\alpha_i) \right] u_t^{(m)}(t) \quad (34)$$

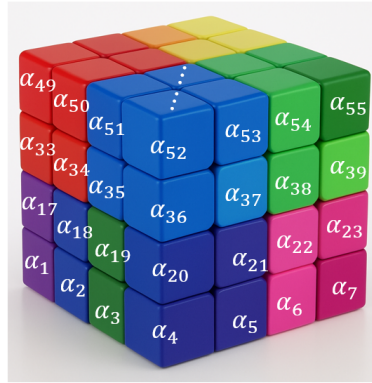


Figure 12: 3D parametric material design problem.

As can be seen from Eq. 34, this is an inherently high-dimensional problem due to the large number of input parametric variables. When a fine mesh is used for the discretization, standard data-driven methods can be extremely

expensive due to the sheer amount of offline data required to be generated in the large sampling space. As an alternative, TAPS does not require any offline data but directly solves the governing parametric PDE in Eq. 33. However, deriving the corresponding matrix equations can be quite challenging and cumbersome due to the complex algebra involved and the flexible nature of the problem statement, as different numbers of subdomains can be used.

GPT-5 is used to derive the matrix form of each subspace iteration. In the *problem statement* part of the prompt, as shown in Fig. 4, apart from the governing equation, we also define the parametric heterogeneous diffusivity field as:

$$k(x, y, z) = \sum_{r=1}^{D_x D_y D_z} k_r I_r(x, y, z), \quad I_r(x, y, z) = I_\alpha(x) I_\beta(y) I_\gamma(z), \quad (35)$$

where $I_\alpha(x_d)$ represents the indicator function whose value is equal to 1 within the corresponding subdomain and zero elsewhere. GPT-5 successfully derives all the detailed equations from the weak form to the final matrix forms in each subspace iteration by a single shot. For example, the final matrix form for the subspace iteration in the α_q direction is:

$$\left(\sum_{\substack{r=1 \\ r \neq q}}^{D_x D_y D_z} \mathbf{B}_{r \neq q}^{[k_q]} \otimes \mathbf{K}^{[N_{\alpha_q} N_{\alpha_q}]} + \mathbf{B}_{r=q}^{[k_q]} \otimes \mathbf{K}^{[N_{\alpha_q} \alpha_q N_{\alpha_q}]} + \mathbf{B} \otimes \mathbf{K}^{[N_{\alpha_q} N_{\alpha_q}]} \right) \text{vec}(\mathbb{U}^{[\alpha_q]}) = \text{vec}(\mathbb{Q}^{[\alpha_q]}) \quad (36)$$

where the coefficient matrices are defined as follows:

$$\mathbf{B} = \mathbf{X}^{(0)} \odot \mathbf{Y}^{(0)} \odot \mathbf{Z}^{(0)} \odot \left(\bigodot_{i \neq q} \mathbf{G}_i^{(0)} \right) \odot \mathbf{T}^{(1)} \quad (37a)$$

$$\mathbf{B}_{r=q}^{[k_q]} = \mathbf{T}^{(0)} \odot \mathbf{S}_q \odot \left(\bigodot_{i \neq q} \mathbf{G}_i^{(0)} \right) \quad (37b)$$

$$\mathbf{B}_{r \neq q}^{[k_q]} = \mathbf{T}^{(0)} \odot \mathbf{S}_r \odot \mathbf{G}_r^{(1)} \odot \left(\bigodot_{i \neq q, r} \mathbf{G}_i^{(0)} \right) \quad (37c)$$

The detailed matrices involved are defined as follows:

$$\mathbf{G}_i^{(0)} = \mathbb{U}^{[\alpha_i]T} \mathbf{K}^{[N_{\alpha_i} N_{\alpha_i}]} \mathbb{U}^{[\alpha_i]} \quad (38a)$$

$$\mathbf{G}_i^{(1)} = \mathbb{U}^{[\alpha_i]T} \mathbf{K}^{[N_{\alpha_i} \alpha_i N_{\alpha_i}]} \mathbb{U}^{[\alpha_i]} \quad (38b)$$

$$\mathbf{S}_r = \mathbf{X}_\alpha^{(B)} \odot \mathbf{Y}_\beta^{(N)} \odot \mathbf{Z}_\gamma^{(N)} + \mathbf{X}_\alpha^{(N)} \odot \mathbf{Y}_\beta^{(B)} \odot \mathbf{Z}_\gamma^{(N)} + \mathbf{X}_\alpha^{(N)} \odot \mathbf{Y}_\beta^{(N)} \odot \mathbf{Z}_\gamma^{(B)} \quad (38c)$$

Temporal:

$$\mathbf{T}^{(0)} = \mathbb{U}^{[t]T} \mathbf{K}^{[N_t N_t]} \mathbb{U}^{[t]} \quad (39a)$$

$$\mathbf{T}^{(1)} = \mathbb{U}^{[t]T} \mathbf{K}^{[N_t B_t]} \mathbb{U}^{[t]} \quad (39b)$$

Unweighted spatial masses:

$$\mathbf{X}^{(0)} = \mathbb{U}^{[x]T} \mathbf{M}^{[x]} \mathbb{U}^{[x]} \quad (40a)$$

$$\mathbf{Y}^{(0)} = \mathbb{U}^{[y]T} \mathbf{M}^{[y]} \mathbb{U}^{[y]} \quad (40b)$$

$$\mathbf{Z}^{(0)} = \mathbb{U}^{[z]T} \mathbf{M}^{[z]} \mathbb{U}^{[z]} \quad (40c)$$

Indicator-weighted spatial contractions:

$$\mathbf{X}_\alpha^{(B)} = \mathbb{U}^{[x]T} \mathbf{K}_x^{(\alpha)} \mathbb{U}^{[x]}, \mathbf{X}_\alpha^{(N)} = \mathbb{U}^{[x]T} \mathbf{M}_x^{(\alpha)} \mathbb{U}^{[x]} \quad (41a)$$

$$\mathbf{Y}_\beta^{(B)} = \mathbb{U}^{[y]T} \mathbf{K}_y^{(\beta)} \mathbb{U}^{[y]}, \mathbf{Y}_\beta^{(N)} = \mathbb{U}^{[y]T} \mathbf{M}_y^{(\beta)} \mathbb{U}^{[y]} \quad (41b)$$

$$\mathbf{Z}_\gamma^{(B)} = \mathbb{U}^{[z]T} \mathbf{K}_z^{(\gamma)} \mathbb{U}^{[z]}, \mathbf{Z}_\gamma^{(N)} = \mathbb{U}^{[z]T} \mathbf{M}_z^{(\gamma)} \mathbb{U}^{[z]} \quad (41c)$$

where the indicator-weighted spatial sparse matrices are defined as:

$$\mathbf{M}_x^{(\alpha)} = \int_{\Omega_x} \tilde{\mathbf{N}}^{[x]T}(x) I_\alpha(x) \tilde{\mathbf{N}}^{[x]}(x) dx \quad (42a)$$

$$\mathbf{K}_x^{(\alpha)} = \int_{\Omega_x} \tilde{\mathbf{B}}^{[x]T}(x) I_\alpha(x) \tilde{\mathbf{B}}^{[x]}(x) dx \quad (42b)$$

We first verify the developed TAPS solver using the following forcing function.

$$f(x, z) = e^{-\left(\frac{x^2}{r_d^2} + \frac{z^2}{r_d^2}\right)} \quad (43)$$

Since there is no analytical solution, the results of the TAPS solver are compared with the full-order FEM simulation using Abaqus. In this comparison, the original domain is composed of $1 \times 2 \times 2$ subdomains. The whole spatial domain is discretized with $80 \times 80 \times 80$ elements. The time t and each diffusivity parameter α_p are also discretized using 80 elements in TAPS. In terms of computational time, Abaqus takes around 1,860 seconds on the Intel Core i9-14900KF (3.20 GHz) CPU to predict the space-time solution for a specific set of diffusivity parameters, while TAPS takes only 601 seconds for the complete parametric simulation (any combinations of diffusivity parameters in the given range). The prediction of TAPS and the Abaqus result for the final time step are shown in Fig. 13. As can be seen from the figure, the TAPS solution agrees well with Abaqus.

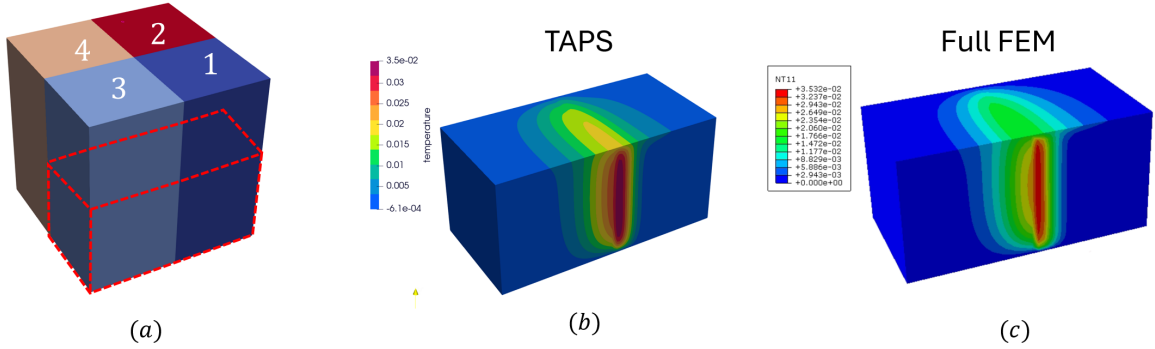


Figure 13: Comparison of TAPS and full FEM prediction.

To rigorously examine the accuracy of the TAPS solver, we use the automated agentic workflow for verification as shown in Fig. 7 and investigate the convergence of the developed TAPS solver. We first adopt a case with $2 \times 2 \times 2$ subdomains. The corresponding convergence of relative L_2 error and computation time plots are shown in Fig. 14. As expected, the parametric TAPS solver has a convergence rate of $p + 1$ for different cases.

Furthermore, we also study the convergence of the TAPS solver for the case with $4 \times 4 \times 4$ subdomains. From Eq. 34, the total number of input variables equals 68, which makes the full-order model extremely high-dimensional. However, the TAPS solver not only has the expected convergence rate, but is also significantly faster. When 200 grid points are used in all dimensions (the equivalent full-order model has 3×10^{156} DoFs), the solution time is around 1,100 seconds. This certifies the efficiency, accuracy, and flexibility of the LLM-empowered TAPS solver for large-scale, high-dimensional parametric problems.

5. Discussion

In the previous sections, we have shown the advantages of the LLM-empowered TAPS solver compared to standard numerical algorithms such as FEM in terms of computational cost. This new approach also has significant benefits in terms of RAM efficiency and disk storage requirements since the reduced-order model has fewer unknowns compared to the full-order model. Moreover, we also demonstrated that LLMs can be used to streamline the development of the TAPS solver for different complex parametric PDEs from a simple 1D S-P-T problem. This remarkable generalization

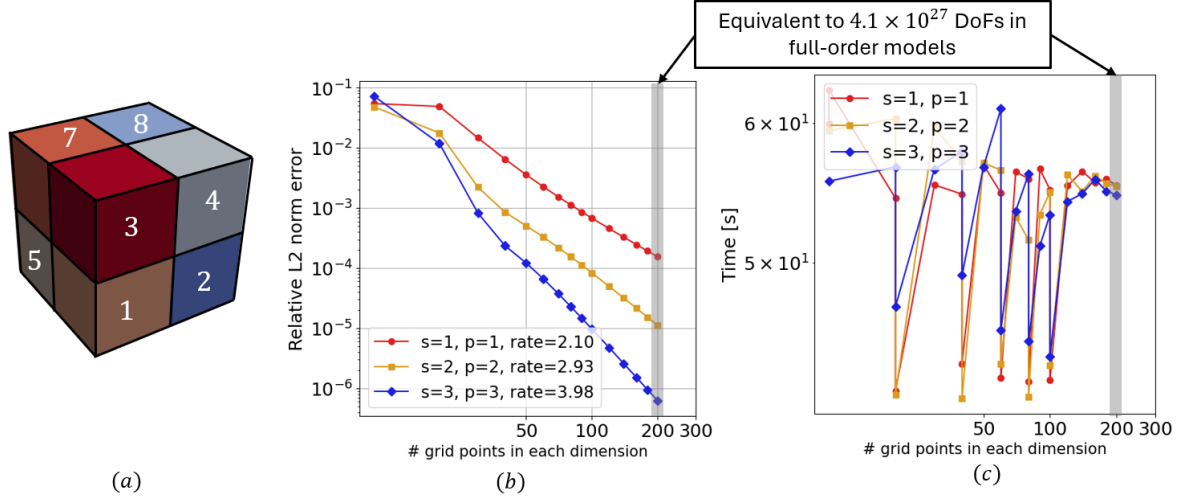


Figure 14: Convergence and solution time for the case with $2 \times 2 \times 2$ subdomains.

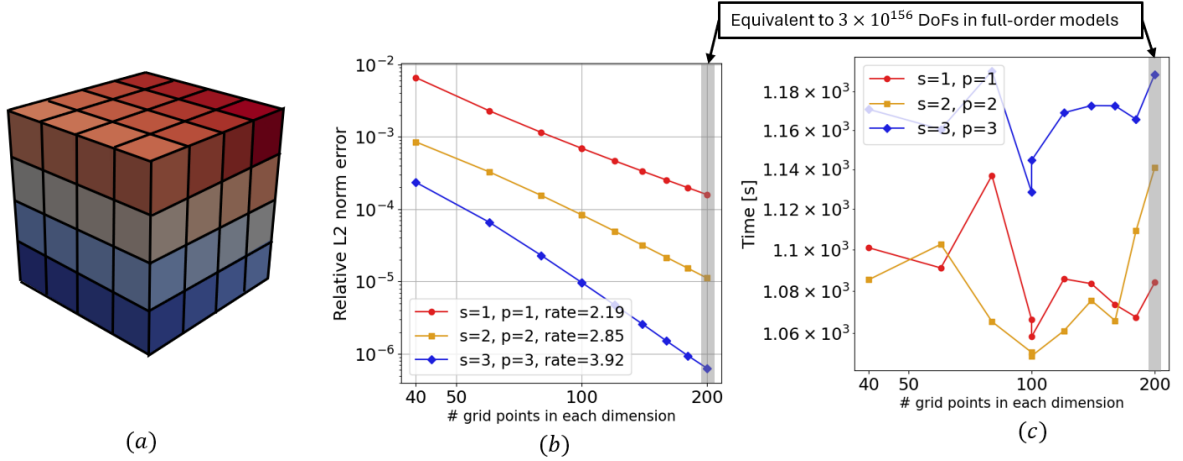


Figure 15: Convergence and solution time for the case with $4 \times 4 \times 4$ subdomains.

capability stems from the LLM's vast pre-trained knowledge of sophisticated numerical algorithms, allowing it to synthesize novel, complex solvers from simple examples.

Compared to standard data-driven surrogates, our new approach does not require offline training data. Consequently, it has superior efficiency for large-scale problems where each full-order simulation data point can be costly to generate for data-driven methods. Moreover, TAPS is built on a weak form and has a controllable convergence rate according to the C-HiDeNN hyperparameters. Therefore, it can achieve orders of magnitude better accuracy compared to data-driven methods, which are probabilistic in nature.

In all the numerical examples, we used TAPS as the data-free MOR solver for ultra-fast, large-scale simulation of parametric PDEs. Nevertheless, note that the proposed framework is versatile and applicable to other intrusive MOR methods, such as proper orthogonal decomposition (POD), provided that a well-formatted example template derivation file and implementation code are included in the prompt for mathematical derivation.

6. Conclusion

In this paper, we propose a new LLM-empowered CAE agent for data-free model order reduction. This new paradigm leverages the power of LLMs and the efficiency of a data-free MOR method: TAPS. LLMs enable an automated workflow from the mathematical derivation, implementation, and verification of the TAPS solver for various kinds of parametric PDEs with minimal human intervention. This new framework can largely alleviate the challenges from the implementation of an intrusive MOR solver to different high-dimensional parametric problems, while ensuring the speed and accuracy of the developed solver. Therefore, it stands out as a promising tool for future CAE agents for ultra-fast, large-scale parametric problems that are common in engineering applications.

In the future, a mechanistic language model (MLM) can be developed by improving the general LLM with domain knowledge in the field of computational science and engineering. This can be done by fine-tuning the pre-trained language model on a curated, high-quality dataset that encompasses continuum mechanics, numerical analysis, tensor algebra, constitutive modeling, high-performance computing, and many others. Moreover, rather than using a standard gigantic transformer architecture as in general-purpose LLMs, the MLM should be of relatively smaller size since it has a relatively narrow focus on the area of CAE. With a much smaller model size, MLM will greatly improve reasoning speed, enabling near real-time mathematical derivation for time-sensitive tasks. Moreover, it can significantly reduce hardware requirements, making it more economical and applicable to typical computing resources [41].

CRedit authorship contribution statement

Jiachen Guo: Writing – original draft, Conceptualization, Methodology, **Chanwook Park:** Writing – review and editing, Data visualization. **Dong Qian:** Writing – review and editing, Project administration, Methodology. **TJR Hughes:** Writing – review and editing, Conceptualization. **Wing Kam Liu:** Writing – review and editing, Project administration, Conceptualization, Methodology.

Appendix A. CAE agent via model context protocol

The Model Context Protocol (MCP) [42] is a recently proposed standard that enables LLMs to interact consistently with external tools and resources through structured messages. In contrast to simple prompt-based augmentation, MCP provides a schema-driven communication layer that ensures reproducibility, extensibility, and interoperability across diverse domains. By exposing software functionalities as MCP tools, developers can allow LLMs to dynamically discover, invoke, and combine these tools during reasoning or workflow execution. In the context of CAE, MCP makes it possible to encapsulate functions such as geometry import, mesh generation, solver execution, and surrogate modeling within a unified framework that LLMs can orchestrate autonomously.

At a practical level, the configuration of a public MCP server allows a desktop LLM agent to access specialized tools. Three common cases illustrate this process:

- **Claude Desktop:** Anthropic’s Claude client natively supports MCP, so a server can be registered by adding the server endpoint to the configuration file. Once linked, Claude can automatically query the available tools and expose them in its interface.
- **Ollama:** Ollama, when extended with an MCP bridge, can connect to local or remote MCP servers by launching with the proper `–mcp-server` flag. This enables LLMs running locally to call MCP tools for simulation, data processing, or visualization without relying on cloud resources.

Through these configurations, MCP transforms LLM agents from text-based assistants into programmable operators of domain-specific workflows. Developing a customized MCP server involves exposing existing computational functionalities as standardized endpoints that an LLM can discover and invoke. At its core, an MCP server acts as a middleware layer: it listens to structured requests from an LLM, executes the corresponding tool or script, and returns results in a machine-readable format. This abstraction allows LLMs to interact with complex software pipelines without directly managing their low-level commands.

A typical MCP server project follows a modular architecture (visually illustrated in A.16):

- **Server layer** (`server.py`): It launches the server, registers tools, and defines available endpoints. When the tools are registered, detailed contexts for each tool should be provided.
- **Modelfile**: This file contains system prompts that contain foundational instructions for an AI model’s behavior, capabilities, and operational constraints before any user interaction occurs.
- **Adapter layer** (`adapters/`): This is where the real work happens. It contains specialized modules (i.e., `tool_N.py`) that know how to interact with different engineering tools (i.e., `source_for_tool_N.py`). Each tool is annotated with metadata that describes its purpose and input/output requirements. This folder also contains the `orchestrator.py` tool, which serves as a central reasoning agent that can plan the usage of multiple tools in this adapter layer from the user query and manage their execution.
- **Source scripts** (`source/`): This folder contains real CAE-related functions (i.e., `source_for_tool_N.py`) that are hard-coded by human experts or other LLMs. The `tool_N.py` modules in the adapter folder configure input arguments from the user prompt and call the correct functions in this folder.

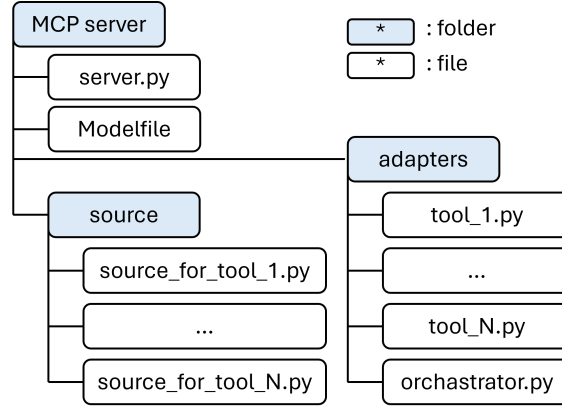


Figure A.16: Architecture of a customized Model Context Protocol (MCP) server. The server exposes computational tools through a standardized interface. Each tool can be implemented through adapters that can call functions in the source folder with correct configurations.

This modular design supports scalability, that is, new tools can be added incrementally without disrupting existing functionality, and engineers can tailor the server to specific CAE tasks. By encapsulating domain expertise within MCP tools, the server effectively becomes a knowledge repository that LLMs can autonomously explore and apply to new problems.

Appendix B. C-HiDeNN-TD approximation

Leveraging the universal approximation theorem, MLP has been widely adopted as a global basis function in deep learning-based solvers [43]. However, MLP exhibits potential caveats when approximating PDE solutions. For example, MLP enforces initial/boundary conditions indirectly by using a penalty term in the loss function, an approach that cannot guarantee that the conditions will be met exactly. Moreover, despite its differentiable nature, the numerical integration of MLP is not straightforward and is more expensive than standard Gaussian integration. As a result, most MLP PDE solvers use collocation methods and rely on stochastic optimization methods to minimize the PDE residual. This means that the convergence and stability of the solution are not guaranteed, leading to potential unreliability.

To overcome these potential caveats, we leverage a novel AI-enhanced basis function called the Convolution Hierarchical Deep-learning Neural Network (C-HiDeNN). This approach marries the merits of both locally supported finite element shape functions and the flexibility of machine learning. Consequently, C-HiDeNN maintains all the essential properties of finite element approximation, such as the Kronecker delta and the partition of unity [12]. Numerical integration of C-HiDeNN basis functions is also straightforward with Gaussian quadrature. As a result, the weak form of the PDE can be adopted similarly to the standard finite element.

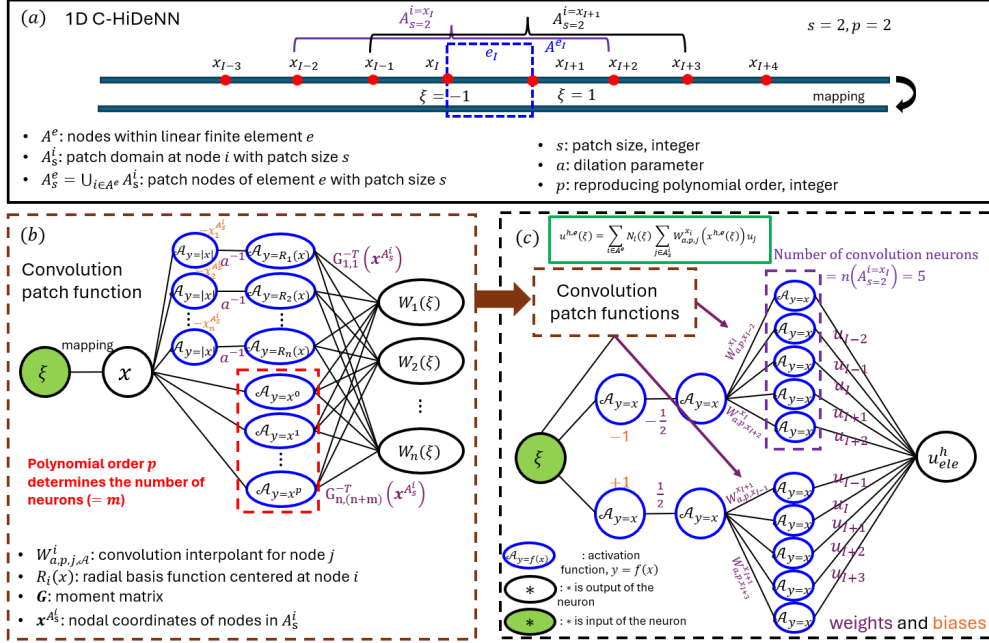


Figure B.17: (a) Covolution patch in 1D C-HiDeNN shape (basis) function (b) Construction of convolution patch function (c) C-HiDeNN shape function as MLP with 3 hidden layers, adapted from [12]

The detailed formulation of the C-HiDeNN basis function is shown in B.17. With C-HiDeNN, a 1D scalar field $u(x)$ defined in each element within a domain Ω_x can be approximated using the C-HiDeNN interpolation as:

$$u_e^h(x) = \sum_{i \in A^e} N_i(x) \sum_{j \in A_s^i} \mathcal{W}_{x^*, a, s, p, j, \mathcal{A}}^i(x) u_j = \sum_{k \in A_s^e} \tilde{N}_k(x; x^*, a, s, p, \mathcal{A}) u_k \quad (\text{B.1})$$

where u_j is the nodal value and $u_j = u(x_j)$; $N_i(x)$ is the linear finite element shape function at node i ; $\mathcal{W}_{x^*, a, s, p, j, \mathcal{A}}^i$ is the convolution patch function at node j for the i -th nodal patch, which can be represented as a partially connected (pruned) MLP, as illustrated in Fig. B.17(b). Note that the weights and biases of the “pruned neural network” are replaced by the nodal positions x^* and the hyperparameters. The convolution patch functions are controlled by the activation function $\mathcal{A}$ and three hyperparameters: patch size s which controls nodal connectivity; the dilation parameter a which normalizes distances between patch nodes; and reproducing order p which defines the types/orders of activation functions to be reproduced by the patch functions. As a result, the univariate function can be approximated as:

$$u^h(x) = \sum_{k=1}^{nnode} \tilde{N}_k(x; x^*, a_k, s_k, p_k, \mathcal{A}_k) u_k \quad (\text{B.2})$$

where $nnode$ is the total number of nodes and k is the nodal index. It should be noted that hyperparameters x^*, a, s, p , and activation $\mathcal{A}$ can vary between different nodal patches since C-HiDeNN can optimize these hyperparameters such as machine learning parameters, rendering an adaptable functional space without altering the number of global nodes or hidden layers.

For ease of discussion, in the remaining part, we fix the hyperparameters x^*, a_k, s_k, p_k and the activation function $\mathcal{A}_k$ in C-HiDeNN, and omit them in the notation. As a result, we obtain the C-HiDeNN-TD approximation of a univariate function.

$$u^h(x) = \sum_{k=1}^{nnode} \tilde{N}_k(x) u_k \quad (\text{B.3})$$

Appendix C. Detailed mathematical derivation of TAPS

In this Appendix, we show the detailed mathematical derivation of TAPS for a 1D space, 1D parameter and 1D time problem. The derivation of this problem serves as the detailed template for the mathematical derivation of other more complex parametric PDEs. The parametric transient heat transfer equation is shown below.

$$\frac{\partial u}{\partial t} - \frac{\partial}{\partial x} \left(\alpha \frac{\partial u}{\partial x} \right) = f(x, \alpha, t) \quad (\text{C.1})$$

subject to homogeneous boundary conditions and initial conditions. This equation has 3 independent variables ($D = 3$), i.e., spatial variable x , parametric variable α and temporal variable t . The S-P-T Galerkin weak form of this problem can be written as follows:

$$\int_{\Omega} \delta u \nabla_t u d\Omega + \int_{\Omega} \nabla_x \delta u \cdot \alpha \nabla_x u d\Omega - \int_{\Omega} \delta u f d\Omega = 0 \quad (\text{C.2})$$

The trial function (solution) is approximated using the C-HiDeNN-TD approximation.

$$u(x, k, t) = \sum_{m=1}^M u_x^{(m)}(x) u_{\alpha}^{(m)}(\alpha) u_t^{(m)}(t) \quad (\text{C.3})$$

The corresponding test function is obtained using the variational principle.

$$\delta u(x, \alpha, t) = \underbrace{\sum_{m=1}^M \delta u_x^{(m)}(x) u_{\alpha}^{(m)}(\alpha) u_t^{(m)}(t)}_{\text{spatial variation}} + \underbrace{\sum_{m=1}^M u_x^{(m)}(x) \delta u_{\alpha}^{(m)}(\alpha) u_t^{(m)}(t)}_{\text{parametric variation}} + \underbrace{\sum_{m=1}^M u_x^{(m)}(x) u_{\alpha}^{(m)}(\alpha) \delta u_t^{(m)}(t)}_{\text{temporal variation}} \quad (\text{C.4})$$

As shown in Eq. C.4, the test function consists of 3 variational terms: spatial, parametric, and temporal variations. Each of the variations will contribute to the final matrix form in each subspace iteration. As an example, we first plug Eq. C.3 and the spatial variation term of Eq. C.4 into Eq. C.2 to obtain the S-P-T weak form terms corresponding to spatial variation:

$$\begin{aligned} & \underbrace{\int_{\Omega} \sum_{m=1}^M \sum_{n=1}^M [\nabla \delta u_x^{(m)}(x) \nabla u_x^{(n)}(x) dx] \cdot [u_{\alpha}^{(m)}(\alpha) \alpha u_{\alpha}^{(n)}(\alpha) d\alpha] \cdot [u_t^{(m)}(t) u_t^{(n)}(t) dt]}_{\text{diffusion term}} + \\ & \underbrace{\int_{\Omega} \sum_{m=1}^M \sum_{n=1}^M [\delta u_x^{(m)}(x) u_x^{(n)}(x) dx] \cdot [u_{\alpha}^{(m)}(\alpha) u_{\alpha}^{(n)}(\alpha) d\alpha] \cdot [u_t^{(m)}(t) \nabla_t u_t^{(n)}(t) dt]}_{\text{time derivative term}} - \\ & \underbrace{\int_{\Omega} \sum_{m=1}^M [\delta u_x^{(m)}(x) f(x) dx] \cdot [u_{\alpha}^{(m)}(\alpha) d\alpha] \cdot [u_t^{(m)}(t) dt]}_{\text{forcing term}} \end{aligned} \quad (\text{C.5})$$

1D C-HiDeNN shape functions are used to approximate each univariate function:

$$\begin{aligned} u_d^{(n)}(x_d) &= \tilde{N}_{n'_d}^{[d]}(x_d) u_{n'_d n}^{[d]} \quad (\text{no sum on } d) \\ \delta u_d^{(m)}(x_d) &= \tilde{N}_{n_d}^{[d]}(x_d) \delta u_{n_d m}^{[d]} \quad (\text{no sum on } d) \end{aligned} \quad (\text{C.6})$$

Here we use Einstein summation to simplify the notation. The free index d refers to dimension and $d = x, k$ or t . The gradient of the interpolated variable can be computed using the shape function derivative $\tilde{B}_{n_d}^{[d]}(x_d) = \frac{d\tilde{N}_{n_d}^{[d]}(x_d)}{dx_d}$.

$$\begin{aligned}\nabla_{x_d} u_d^{(n)}(x_d) &= \widetilde{B}_{n'_d}^{[d]}(x_d) u_{n'_d n}^{[d]} \quad (\text{no sum on } d) \\ \nabla_{x_d} \delta u_d^{(m)}(x_d) &= \widetilde{B}_{n_d}^{[d]}(x_d) \delta u_{n_d m}^{[d]} \quad (\text{no sum on } d)\end{aligned}\tag{C.7}$$

Plugging Eqs. C.6 - C.7 into Eq. C.5, the diffusion term in Eq. C.5 can be rewritten as:

$$\sum_{m=1}^M \sum_{n=1}^M \underbrace{\int_{\Omega_x} \widetilde{B}_{n_x}(x) \delta u_{n_x m}^{[x]} \widetilde{B}_{n'_x}(x) u_{n'_x n}^{[x]} dx}_{\text{spatial term}} \cdot \underbrace{\int_{\Omega_\alpha} \widetilde{N}_{n_\alpha}(\alpha) u_{n_\alpha m}^{[\alpha]} \alpha \widetilde{N}_{n'_\alpha}(\alpha) u_{n'_\alpha n}^{[\alpha]} d\alpha}_{\text{parametric term}} \cdot \underbrace{\int_{\Omega_t} \widetilde{N}_{n_t}(t) u_{n_t m}^{[t]} \widetilde{N}_{n'_t}(t) u_{n'_t n}^{[t]} dt}_{\text{temporal term}} \tag{C.8}$$

As can be readily seen from Eq. C.8, after doing 1D integration of each term, the parametric and temporal terms can be treated as coefficient matrices:

$$\begin{aligned}C_{mn}^{[\alpha]} &= \int_{\Omega_\alpha} \widetilde{N}_{n_\alpha}(\alpha) u_{n_\alpha m}^{[\alpha]} \alpha \widetilde{N}_{n'_\alpha}(\alpha) u_{n'_\alpha n}^{[\alpha]} d\alpha \\ C_{mn}^{[t]} &= \int_{\Omega_t} \widetilde{N}_{n_t}(t) u_{n_t m}^{[t]} \widetilde{N}_{n'_t}(t) u_{n'_t n}^{[t]} dt\end{aligned}\tag{C.9}$$

as the only free indices are m and n . Substituting the coefficient matrices and rearranging different terms in Eq. C.8, we have:

$$\sum_{m=1}^M \delta u_{n_x m}^{[x]} \sum_{n=1}^M \left[\int_{\Omega_x} \widetilde{B}_{n_x}(x) \widetilde{B}_{n'_x}(x) dx \right] \cdot C_{mn}^{[\alpha]} C_{mn}^{[t]} \cdot u_{n'_x n}^{[x]} \tag{C.10}$$

Like standard FEM, we can define $\int_x \widetilde{B}_{n_x}(x) \widetilde{B}_{n'_x}(x) dx$ as the 1D stiffness matrix $K_{n_x n'_x}^{[x]}$ of x dimension in Eq. C.10. Furthermore, the following fourth-order tensor can be defined (no sum on m, n):

$$A_{n_x n'_x mn}^{[x]} = K_{n_x n'_x}^{[x]} C_{mn}^{[\alpha]} C_{mn}^{[t]} \tag{C.11}$$

Therefore, Eq. C.8 can be further simplified as follows:

$$\delta u_{n_x m}^{[x]} A_{n_x n'_x mn}^{[x]} u_{n'_x n}^{[x]} \tag{C.12}$$

The 4-th order tensor $A_{n_x n'_x mn}^{[x]}$ can be reshaped as a 2nd order tensor $\mathbb{A}_{IJ}^{[x]}$. The trial and test function solution vectors can also be vectorized:

$$A_{n_x n'_x mn}^{[x]} = \mathbb{A}_{IJ}^{[x]} \tag{C.13a}$$

$$\begin{aligned}\text{vec}(\delta \mathbb{U}^{[x]})_I &= \left[\text{vec} \left(\delta u_{n_x m}^{[x]} \right) \right]_I \\ \text{vec}(\mathbb{U}^{[x]})_J &= \left[\text{vec} \left(u_{n'_x n}^{[x]} \right) \right]_J\end{aligned}\tag{C.13b}$$

As a result, Eq. C.12 can be rewritten in the following matrix form:

$$\text{vec}(\delta \mathbb{U}^{[x]})^T \mathbb{A}^{[x]} \text{vec}(\mathbb{U}^{[x]}) \tag{C.14}$$

Following the same procedure, we can obtain matrix forms corresponding to the time derivative term $\text{vec}(\delta \mathbb{U}^{[x]})^T \mathbb{B}^{[x]} \text{vec}(\mathbb{U}^{[x]})$, and the forcing term $\text{vec}(\delta \mathbb{U}^{[x]})^T \text{vec}(\mathbb{Q}^{[x]})$ in the spatial variational part of Eq. C.5. Similar structures can also be obtained for the parametric and temporal variational parts of the test function in Eq. C.4. Denoting $\mathbb{K}^{[d]} = \mathbb{A}^{[d]} + \mathbb{B}^{[d]}$, the matrix form of the generalized S-P-T Galerkin form in Eq. C.2 can be written as:

$$\underbrace{\text{vec}(\delta \mathbb{U}^{[x]})^T \mathbb{K}^{[x]} \text{vec}(\mathbb{U}^{[x]}) - \text{vec}(\delta \mathbb{U}^{[x]})^T \text{vec}(\mathbb{Q}^{[x]})}_{\text{spatial variational part}} + \underbrace{\text{vec}(\delta \mathbb{U}^{[\alpha]})^T \mathbb{K}^{[\alpha]} \text{vec}(\mathbb{U}^{[\alpha]}) - \text{vec}(\delta \mathbb{U}^{[\alpha]})^T \text{vec}(\mathbb{Q}^{[\alpha]})}_{\text{parametric variational part}} + \underbrace{\text{vec}(\delta \mathbb{U}^{[t]})^T \mathbb{K}^{[t]} \text{vec}(\mathbb{U}^{[t]}) - \text{vec}(\delta \mathbb{U}^{[t]})^T \text{vec}(\mathbb{Q}^{[t]})}_{\text{temporal variational part}} = 0$$

The above equation is a nonlinear system of equations since $\mathbb{K}^{[d]}$ depends on solution vectors in other dimensions. Using subspace iteration, this nonlinear system can be recast into a series of linear systems. For example, in the subspace iteration in the x direction, we let the variation in other dimensions, i.e., $\text{vec}(\delta \mathbb{U}^{[\alpha]})$ and $\text{vec}(\delta \mathbb{U}^{[t]})$ equal to 0. Due to the arbitrariness of the test function vector $\text{vec}(\delta \mathbb{U}^{[x]})$, the above equation becomes:

$$\mathbb{K}^{[x]} \text{vec}(\mathbb{U}^{[x]}) - \text{vec}(\mathbb{Q}^{[x]}) = 0 \quad (\text{C.16})$$

After $\text{vec}(\mathbb{U}^{[x]})$ is obtained, we update the matrix $\mathbb{K}^{[\alpha]}$ and the subspace iteration in the α dimension can be written as:

$$\mathbb{K}^{[\alpha]} \text{vec}(\mathbb{U}^{[\alpha]}) - \text{vec}(\mathbb{Q}^{[\alpha]}) = 0 \quad (\text{C.17})$$

We keep iterating in different dimensions until the variation of the solution vector in each dimension is within the tolerance.

Appendix D. Prompt for mathematical derivation of TAPS solver

In this appendix, we present the complete prompt for the mathematical derivation of new given PDEs for the TAPS solver. As shown in Fig. D.18, it consists of 5 major parts: 1. role-playing, where we set up context for the LLMs; 2. few-shot prompt, where the template mathematical derivation is provided; 3. constraints, by which we apply the matrices that the derivation can use; 4. chain-of-thought, where we enforce LLMs to derive the equations step-by-step; 5. formatting guidelines, by which we instruct LLMs to generate a well-formatted markdown with all equations written in $\text{\LaTeX}$.

References

- [1] Andrej Karpathy. Software 2.0, Mar 2017. URL <https://karpathy.medium.com/software-2-0-a64152b37c35>.
- [2] Andrej Karpathy. Software 3.0, June 2025. URL <https://www.ycombinator.com/library/MW-andrej-karpathy-software-is-changing-again>.
- [3] Mohaimenul Azam Khan Raiaan, Md Saddam Hossain Mukta, Kaniz Fatema, Nur Mohammad Fahad, Sadman Sakib, Most Marufatul Jannat Mim, Jubaer Ahmad, Mohammed Eunus Ali, and Sami Azam. A review on large language models: Architectures, applications, taxonomies, open issues and challenges. *IEEE access*, 12: 26839–26874, 2024.
- [4] Zabir Al Nazi and Wei Peng. Large language models in healthcare and medical domain: A review. In *Informatics*, volume 11, page 57. MDPI, 2024.
- [5] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günemann, Eyke Hüllermeier, et al. Chatgpt for good? on opportunities and challenges of large language models for education. *Learning and individual differences*, 103:102274, 2023.
- [6] Jaeho Jeon and Seongyong Lee. Large language models in education: A focus on the complementary relationship between human teachers and chatgpt. *Education and Information Technologies*, 28(12):15873–15892, 2023.

You are a computational mechanics expert tasked with making targeted corrections to a mathematical derivation.

****TASK**:** Create the mathematical derivation for the given problem statement based on the template.

****INPUT**:**

- Problem statement: {{problem_statement_md_file}}
- Template derivation: {{template_md_file}}
- Matrix definition: {{matrix_md_file}}

****REQUIRED ACTIONS**:**

1. Start with the complete template derivation
2. Adapt the template derivation to the problem statement, derive step by step
3. Keep ALL other content the same
4. Add a brief note explaining the derivation made
5. All matrix definitions should be based on the matrix definition provided.

****IMPORTANT**:**

- Carefully do the symbolic manipulation to ensure the derivation is correct, especially for terms involving the indicator functions.
- *diffusion term*: do the symbolic derivation step by step for every subspace iteration, especially the K direction iteration.
- Carefully do step by step derivation for terms involving the summation with Σ notation and multiplication with Π notation.
- For triple terms like: $\int_{\Omega} \tilde{N}_n(x) \tilde{N}_m(x) dx$, use basis bilinear stiffness matrix.

****OUTPUT FORMAT**:**

- Complete markdown file with the derivation
- Use standard LaTeX math formatting ($\dots$ or $\$...\$$)
- Preserve all original structure, headings, and content

Role-playing

Few-shot prompt

Constraints

Chain-of-thought

Formatting guidelines

Figure D.18: Prompt used for mathematical reasoning to derive the discretized form of different PDEs in the TAPS framework.

- [7] Zhiyu Zoey Chen, Jing Ma, Xinlu Zhang, Nan Hao, An Yan, Armineh Nourbakhsh, Xianjun Yang, Julian McAuley, Linda Petzold, and William Yang Wang. A survey on large language models for critical societal domains: Finance, healthcare, and law. *arXiv preprint arXiv:2405.01769*, 2024.
- [8] Wing Kam Liu, Shaofan Li, and Harold S Park. Eighty years of the finite element method: Birth, evolution, and future. *Archives of Computational Methods in Engineering*, 29(6):4431–4453, 2022.
- [9] Lei Zhang, Lin Cheng, Hengyang Li, Jiaying Gao, Cheng Yu, Reno Domel, Yang Yang, Shaoqiang Tang, and Wing Kam Liu. Hierarchical deep-learning neural networks: finite elements and beyond. *Computational Mechanics*, 67:207–230, 2021.
- [10] Lei Zhang, Ye Lu, Shaoqiang Tang, and Wing Kam Liu. Hidenn-td: Reduced-order hierarchical deep learning neural networks. *Computer Methods in Applied Mechanics and Engineering*, 389:114414, 2022.
- [11] Ye Lu, Hengyang Li, Lei Zhang, Chanwook Park, Satyajit Mojumder, Stefan Knapik, Zhongsheng Sang, Shaoqiang Tang, Daniel W Apley, Gregory J Wagner, et al. Convolution hierarchical deep-learning neural networks (c-hidenn): finite elements, isogeometric analysis, tensor decomposition, and beyond. *Computational Mechanics*, pages 1–30, 2023.
- [12] Chanwook Park, Ye Lu, Sourav Saha, Tianju Xue, Jiachen Guo, Satyajit Mojumder, Daniel W Apley, Gregory J Wagner, and Wing Kam Liu. Convolution hierarchical deep-learning neural network (c-hidenn) with graphics processing unit (gpu) acceleration. *Computational Mechanics*, 72(2):383–409, 2023.
- [13] Hengyang Li, Stefan Knapik, Yangfan Li, Chanwook Park, Jiachen Guo, Satyajit Mojumder, Ye Lu, Wei Chen, Daniel W Apley, and Wing Kam Liu. Convolution hierarchical deep-learning neural network tensor decomposition (c-hidenn-td) for high-resolution topology optimization. *Computational Mechanics*, pages 1–20, 2023.

- [14] Chanwook Park, Sourav Saha, Jiachen Guo, Hantao Zhang, Xiaoyu Xie, Miguel A Bessa, Dong Qian, Wei Chen, Gregory J Wagner, Jian Cao, et al. Interpolating neural network: A novel unification of machine learning and interpolation theory. *arXiv preprint arXiv:2404.10296*, 2024.
- [15] Jiachen Guo, Xiaoyu Xie, Chanwook Park, Hantao Zhang, Matthew Politis, Gino Domel, T.J.R Hughes, and Wing Kam Liu. Interpolation neural network-tensor decomposition (inn-td): a scalable and interpretable approach for large-scale physics-based problems. *arXiv preprint arXiv:2503.02041*, 2025.
- [16] Jiachen Guo, Gino Domel, Chanwook Park, Hantao Zhang, Ozgur Can Gumus, Ye Lu, Gregory J Wagner, Dong Qian, Jian Cao, Thomas JR Hughes, et al. Tensor-decomposition-based a priori surrogate (taps) modeling for ultra large-scale simulations. *Computer Methods in Applied Mechanics and Engineering*, 444:118101, 2025.
- [17] Prashant Govindarajan, Davide Baldelli, Jay Pathak, Quentin Fournier, and Sarath Chandar. Cadmium: Fine-tuning code language models for text-driven sequential cad design. *arXiv preprint arXiv:2507.09792*, 2025.
- [18] Chaofan Lv and Jinsong Bao. Cadinstruct: A multimodal dataset for natural language-guided cad program synthesis. *Computer-Aided Design*, page 103926, 2025.
- [19] Licheng Zhang, Bach Le, Naveed Akhtar, Siew-Kei Lam, and Tuan Ngo. Large language models for computer-aided design: A survey. *arXiv preprint arXiv:2505.08137*, 2025.
- [20] Shifu Hou, Rick Johnson, Ramandeep Makhija, Lingwei Chen, and Yanfang Ye. Autofea: Enhancing ai copilot by integrating finite element analysis using large language models with graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24078–24085, 2025.
- [21] Nayantara Mudur, Hao Cui, Subhashini Venugopalan, Paul Raccuglia, Michael Brenner, and Peter Christian Norgaard. Feabench: Evaluating language models on real world physics reasoning ability. 2024.
- [22] Sandeep Pandey, Ran Xu, Wenkang Wang, and Xu Chu. Openfoamgpt: A retrieval-augmented large language model (llm) agent for openfoam-based computational fluid dynamics. *Physics of Fluids*, 37(3), 2025.
- [23] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.
- [24] Mikko Lehtimäki, Lassi Paunonen, and Marja-Leena Linne. Accelerating neural odes using model order reduction. *IEEE Transactions on Neural Networks and Learning Systems*, 35(1):519–531, 2022.
- [25] Mojtaba Mozaffar, Ramin Bostanabad, W Chen, K Ehmann, Jian Cao, and MA Bessa. Deep learning predicts path-dependent plasticity. *Proceedings of the National Academy of Sciences*, 116(52):26414–26420, 2019.
- [26] Francisco Chinesta, Roland Keunings, and Adrien Leygue. *The proper generalized decomposition for advanced numerical simulations: a primer*. Springer Science & Business Media, 2013.
- [27] Jiachen Guo, Chanwook Park, Xiaoyu Xie, Zhongsheng Sang, Gregory J Wagner, and Wing Kam Liu. Convolutional hierarchical deep learning neural networks-tensor decomposition (c-hidenn-td): a scalable surrogate modeling approach for large-scale physical systems. *arXiv preprint arXiv:2409.00329*, 2024.
- [28] Yichen Huang and Lin F Yang. Gemini 2.5 pro capable of winning gold at imo 2025. *arXiv preprint arXiv:2507.15855*, 2025.
- [29] Yuri Chervonyi, Trieu H Trinh, Miroslav Olšák, Xiaomeng Yang, Hoang Nguyen, Marcelo Menegali, Junehyuk Jung, Vikas Verma, Quoc V Le, and Thang Luong. Gold-medalist performance in solving olympiad geometry with alpageometry2. *arXiv preprint arXiv:2502.03544*, 2025.
- [30] Michael Shalyt, Rotem Elimelech, and Ido Kaminer. Asymob: Algebraic symbolic mathematical operations benchmark. *arXiv preprint arXiv:2505.23851*, 2025.

- [31] Xinyi Hou, Yanjie Zhao, Shenao Wang, and Haoyu Wang. Model context protocol (mcp): Landscape, security threats, and future research directions. *arXiv preprint arXiv:2503.23278*, 2025.
- [32] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [33] Gal Berkooz, Philip Holmes, and John L Lumley. The proper orthogonal decomposition in the analysis of turbulent flows. *Annual review of fluid mechanics*, 25(1):539–575, 1993.
- [34] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate, 2025. URL <https://arxiv.org/abs/2509.04664>.
- [35] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [36] Oguzhan Topsakal and Tahir Cetin Akinci. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International conference on applied engineering and natural sciences*, volume 1, pages 1050–1056, 2023.
- [37] Tomislav Maric, Dennis Gläser, Jan-Patrick Lehr, Ioannis Papagiannidis, Benjamin Lambie, Christian Bischof, and Dieter Bothe. A research software engineering workflow for computational science and engineering. *arXiv preprint arXiv:2208.07460*, 2022.
- [38] Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.
- [39] Reem Aleithan, Haoran Xue, Mohammad Mahdi Mohajer, Elijah Nnorom, Gias Uddin, and Song Wang. Swe-bench+: Enhanced coding benchmark for llms. *arXiv preprint arXiv:2410.06992*, 2024.
- [40] Justin D Weisz, Shraddha Vijay Kumar, Michael Muller, Karen-Ellen Browne, Arielle Goldberg, Katrin Ellice Heintze, and Shagun Bajpai. Examining the use and impact of an ai code assistant on developer productivity and experience in the enterprise. In *Proceedings of the Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2025.
- [41] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. Small language models are the future of agentic ai. *arXiv preprint arXiv:2506.02153*, 2025.
- [42] Anthropic, OpenAI, LangChain, and Others. Model context protocol (mcp): A standard for connecting llms to tools and data, 2023. URL <https://modelcontextprotocol.io>. Accessed: 2025-09-08.
- [43] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational physics*, 378:686–707, 2019.